

The Australian National University
2600 ACT | Canberra | Australia



Australian
National
University

School of Computing

College of Engineering, Computing
and Cybernetics (CECC)

Combining Structural Heuristics with Neural Networks for Intelli- gent Instruments

— Honours project (S2/S1 2024–2025)

A thesis submitted for the degree

Bachelor of Advanced Computing (Research & Development) (Honours)

By:

Scott Howsam

Supervisor:

Charles Martin

April 2025

Declaration:

I declare that this work:

- upholds the principles of academic integrity, as defined in the [University Academic Misconduct Rules](#);
- is original, except where collaboration (for example group work) has been authorised in writing by the course convener in the class summary and/or Wattle site;
- is produced for the purposes of this assessment task and has not been submitted for assessment in any other context, except where authorised in writing by the course convener;
- gives appropriate acknowledgement of the ideas, scholarship and intellectual property of others insofar as these have been used;
- in no part involves copying, cheating, collusion, fabrication, plagiarism or recycling.

April, Scott Howsam

Abstract

TODO

Table of Contents

1	Introduction	1
1.1	Research Questions	3
1.2	Methodology	4
1.3	Thesis Structure	5
2	Background	7
2.1	New Interfaces for Musical Expression (NIMEs)	8
2.1.1	Controller Mappings	8
2.1.2	Collaborative Networked Instruments	10
2.1.3	Interactive Musical Prediction Systems	10
2.2	Machine Learning in Music	11
2.2.1	Markov Models	11
2.2.2	Feedforward Neural Networks	12
2.2.3	Recurrent Neural Networks	13
2.2.4	Long Short Term Memory Networks	15
2.2.5	Mixture Density Recurrent Neural Networks	17
2.3	Heuristic-Based AI in Music	18
2.3.1	Rules Based Systems	19
2.3.2	Search Based Systems	20
2.4	Tree Search Algorithms	21
2.4.1	Beam Search	21
2.4.2	Monte Carlo Tree Search	22
2.4.3	Continuous Monte Carlo Tree Search	23
2.5	Applications of Combined Machine Learning and Search	26
2.5.1	Game Playing AI	26
2.5.2	Natural Language Processing	26
2.5.3	Robot Motion Planning	27
2.6	IMPSY	28
3	System Design	31
3.1	Design Overview	31

Table of Contents

3.2	Monte-Carlo Tree Search	31
3.2.1	Continuous Monte-Carlo Tree Search	32
3.3	The Successor Algorithm	33
3.4	Heuristics	33
3.4.1	Setting Heuristics Through Human Performance	36
3.4.2	Identifying Patterns of Repetition	37
3.4.3	Utilising Heuristics for Prediction	37
3.5	Performance Parameters	37
3.5.1	Tempo Parameter	37
3.5.2	EACH OF THE OTHER PARAMETERS GOES IN A SUBSEC- TION HERE	38
3.6	Human Interaction	38
3.6.1	Performance Parameter User Interface	38
3.7	Training New Performance Parameters	38
4	Experiments	39
5	Evaluation	41
5.1	Tuning Metaparameters	41
5.2	Quantitative Evaluation	41
5.3	Qualitative Evaluation	42
6	Discussion	43
6.1	Limitations	43
6.2	Future Work	43
7	Conclusion	45
	Bibliography	47

Introduction

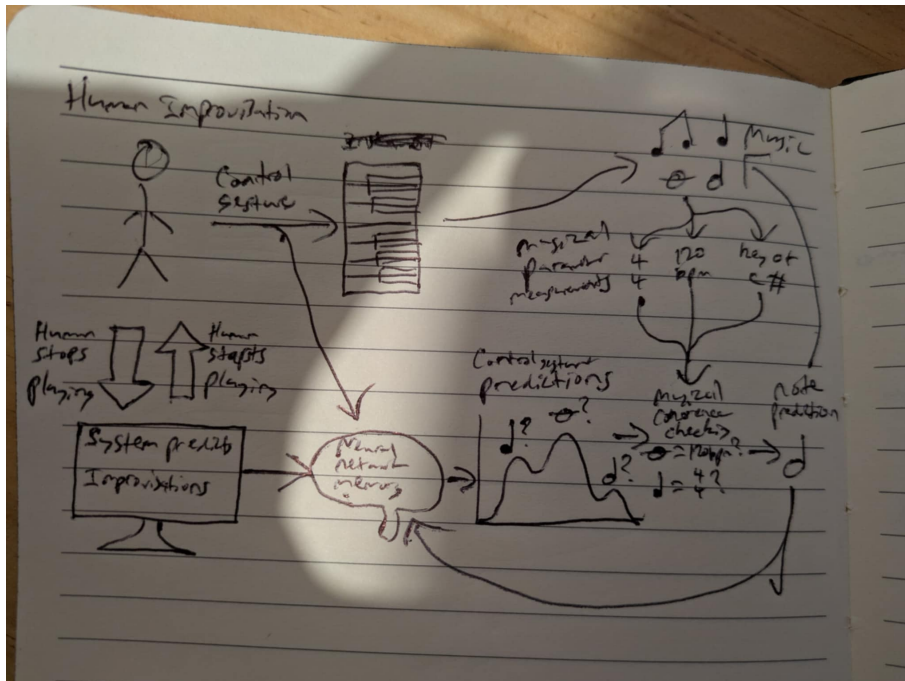


Figure 1.1: Our interactive musical prediction system can gracefully take over from human improvisation using a neural network and musical coherence checking.

New Interfaces for Musical Expression (NIMEs) are an important field of research for making musical interfaces more accessible, expressive, creative, and fun. In real-time musical performance, NIMEs provide new ways for musicians to interact with digital systems, expanding the possibilities for improvisation, collaboration, and musical exploration. Music is a fundamental aspect of human culture and self-expression, and

1 Introduction

improving the ways people can interact with it can increase inclusion and participation in this important aspect of human culture. Additionally, as AI-driven creative tools become more sophisticated, they have the potential to redefine artistic collaboration between humans and machines, opening new pathways for artistic expression. By improving the capabilities of NIMEs, we can not only enhance artistic possibilities, but also contribute to a deeper, more accessible, and more human-centered approach to digital music interaction. By augmenting or transforming traditional instruments, or even introducing entirely new modes of interaction, NIMEs allow performers to push the boundaries of human-computer interaction in music. Real-time interactive NIMEs are a subgroup which can interact with human performers in expressive ways, making them particularly valuable in live performance contexts. However, designing a system that can both understand and contribute meaningfully to a live musical performance remains a challenging task, requiring a balance between structured musical coherence and the flexibility of creative improvisation. This is a problem when developing interactive musical prediction systems which can improvise with a human in a call-and-response fashion, and is the focus of our research.

Recent developments in NIMEs which can be applied in real-time musical performance have broadly fallen into two categories. Heuristic based approaches break down musical harmony and structure symbolically into discrete components which can be logically reasoned with. By converting the expression of a human performer into this symbolic language, a NIME can then dynamically interact with the performer by performing calculations in this logical space (Fu et al., 2023). This includes rule-based methods, where hardcoded musical constraints direct composition, and search based techniques, which explore the space of future musical directions, while being guided by heuristics. These methods are excellent for generating structured, coherent, and theoretically sensical musical outputs. However, they suffer from restrictions on the breadth and complexity of musical expression, which often leads to robotic and less creative sounding results (Anders and Miranda, 2011).

In contrast, other methods have used recurrent neural networks (RNNs) trained on human performance to repeatedly predict the most likely next step in a performance. When used for NIMEs, these techniques solve many of the issues of heuristic-based approaches, as they are formulated to continuously predict the most human sounding continuation of a musical performance. However, these approaches aim for only the most likely next step, often leading to human-like, but very predictable and repetitive musical outputs. These is usually mitigated by adding randomness to the prediction process, by way of increased temperature, softmax sampling, or random sampling using Mixture Density Networks (MDNs). While adding randomness to the prediction process can make the performance more creative, it is a very arbitrary way to add this creativity and diversity, and has the tendency to very quickly lead to incoherence if the randomness is too high (Martin and Tørresen, 2018, 2019).

This motivates the question of whether a more guided approach than the addition of pure randomness to the prediction process could be used, and such a guided approach

is what we present in this thesis. Our hybrid system, shown in Figure 1.1, uses both heuristic and mixture density recurrent neural network (MDRNN) based approaches, with a search algorithm being used to guide the choice of outputs from the gaussian mixture model (GMM) of output likelihoods provided by an MDRNN prediction. From making repeated MDRNN predictions, and treating these predictions as a domain of actions within monte carlo tree search, a tree of possible simulated musical continuations is generated. Then, from the measurement of structural parameters of the music, such as harmonic consonance, rhythmic uniformity, and pitch variation (TODO: make these our actual parameters once the system design is complete), a heuristic measures the level of conformity in each simulated musical continuation to important musical qualities of the piece. Finally, these heuristic measurements are backpropogated through the search tree and used to guide the next musical output toward a choice which is consistent with the previous state of the performance. This allows for the system to maintain the benefit of MDRNNs, which is the ability to generate more creative and human-like outputs, without sacrificing musical coherence, and in this way better imitates the thought process of a human performer.

We apply our research method to the development of a new prediction algorithm for IMPSY (Martin and Tørresen, 2019), an interactive musical prediction system which uses an MDRNN to continue human improvisation in a call-and-response manner, aiming to generate improvisation which coherently carries on from the human performer. IMPSY is notably unique in that it predicts control gestures for an instrument, instead of the musical output itself. Currently, IMPSY randomly samples from a GMM output at each step to mitigate the predictability standard RNNs. Our research aims to guide this sampling instead through the method we have described.

1.1 Research Questions

This thesis aims to address the following questions:

1. What are the strengths and weaknesses of heuristic and machine learning based approaches to NIMEs?

In Chapter 2, we will review the literature on NIMEs, including both heuristic and machine learning based approaches. We will discuss the strengths and weaknesses of each approach, and how they can be combined to create a more effective system.

2. Are similar approaches to our research being applied successfully in other fields?

Chapter 2 will also look into other fields where combinations of heuristic and machine learning based approaches have been successfully applied. This includes discussion of some specific papers with very similar approaches to ours, using MDRNNs to generate GMM outputs, and then using those GMM outputs as continuous action domains for MCTS.

3. How can a combined MDRNN and MCTS algorithm be implemented into IMPSY?

In Chapter 3, we will describe our implementation of the combined MDRNN and MCTS algorithm, and its integration into IMPSY. We will discuss the details of the algorithm, including each of our design choices and why we believe they are the best choices for our system.

4. What are the most important musical parameters for maintaining coherence, and how can these be quantified into a heuristic?

Chapter 3 will also describe the heuristic we have developed for measuring the coherence of a musical performance. Each musical parameter which goes into this heuristic is described in detail, including how it is calculated and why it is important.

5. Which meta-parameters are important to the performance of our system, and how can they be tuned optimally?

Chapter 4 describes the experiments we have conducted to test our system and fine tune meta-parameters important to the algorithms performance. We will discuss the results of these experiments, and how they affect fine tuning of the system.

6. How does our system compare to existing approaches in terms of musical quality and improvisational prediction ability?

In Chapter 5, we will evaluate our system using both quantitative and qualitative measures. We will compare our system to existing approaches, including IMPSY, for measures of both musical quality and improvisational prediction ability. In Chapter 6 and 7 we then also discuss the implications of our research for the future of NIMEs and interactive musical performance.

1.2 Methodology

TODO: this is a very old draft of this section and doesn't make much sense anymore, however I don't think it's worth updating until I finish a draft of the system design, experiments, and evaluation sections. Skip this section when providing feedback.

Our combined approach for developing real-time interactive NIMEs uses an RNN based IMPS as a baseline algorithm, and extends it with a symbolic structural heuristic. As a baseline algorithm we use the mixture density recurrent neural network (MDRNN) IMPS developed by (cite An Interactive Musical Prediction System with Mixture Density Recurrent Neural Networks), which uses an RNN based approach. We then modify the prediction system. At each point where a prediction must be made, instead of outputting the first prediction made by the RNN, the RNN is run multiple times recursively, generating an expanding tree of possible branches the performance could continue along while sounding human like. On each of these branches, we then measure a set of structural parameters. Examples of these parameters are rhythmic uniformity, harmonic

consonance, and pitch variation. Branches which are more consistent with the previous state of these structural parameters are given higher heuristic values, leading the choice of next prediction to favour musical directions which don't break from the long term structural norm in an incoherent way. When interacting with a human performer, these structural parameters, and the rate of change of each, will be tracked and used to inform the AI when it takes over from the human in the performance, ensuring that it continues the performance in the same structural direction that the human performer was moving in. This also allows for sudden breaks from structural parameters in places where it is deemed to be a reasonable direction for the piece, for example the beginning of a bridge section. We also present an interface for the human performer to control the structural parameters of the AI while it plays in a live setting, and for new and unique structural parameters to be trained from human performance, allowing for longer term parameters which are difficult to symbolically code, such as the mood of a piece, to be encoded using machine learning instead.

Using RNN predictions to determine on a short time scale which musical directions are appropriate ensures that the performance will sound more human and creative than symbolic approaches. The addition of symbolically encoded structural parameters and a tree search based system of choosing the musical direction of a piece negates the disadvantages of RNN based approaches, as it provides a framework for coherent long term structure, and balances out the greedy next step prediction process of a purely RNN based approach.

1.3 Thesis Structure

This thesis consists of 7 chapters. After this introduction in Chapter 1, Chapter 2 covers the necessary background information which is important for understanding our research in context. This includes a brief overview of NIMEs more generally, as well as the development of both machine learning and heuristic based approaches to NIMEs. It also covers combinations of these methods which are similar to our research, but applied in different fields. Finally, it discusses IMPSY, the system our research expands upon, and its limitations which we aim to address. Chapter 3 details our system design and implementation process, including the novel algorithm we have developed for predicting the next step in a musical performance, and the details of each component in the heuristic used in this algorithm. Chapter 4 describes the experiments we have conducted to test our system and fine tune meta-parameters important to the algorithms performance. Chapter 5 then details both quantitative and qualitative measures of evaluation we have used to assess the performance of our system. Chapter 6 discusses the results of our experiments and evaluation, comparing our system to existing approaches and noting its strengths, limitations, and future work which could be conducted to expand upon our research. Finally, Chapter 7 concludes the thesis by summarising our findings and discussing the implications of our research.

Background

This thesis relates to work in multiple different fields, including musical controllers, generative music, and robot motion planning, and this chapter provides an overview of each of these fields, connecting back to their relevance to our research. It will cover the advantages and disadvantages of various AI methods and their applications to computer music, and discuss gaps in research which we aim to cover within this thesis. This chapter also covers the machine learning and search algorithms used in our system design, their previous applications, and their potential to be applied within our research. We begin with an overview of New Interfaces for Musical Expression (NIMEs), including a brief history of developments in computer music which led to the creation of the field, important NIME papers, and the major subclasses of NIMEs. We then discuss how machine learning has been used in music, and explore some different types of neural networks and their relevance to our research. This is followed by a look into heuristic based AI, a different approach than machine learning based techniques, and its advantages and disadvantages in music generation. Next, the background for search methods not commonly used in previous computer music literature, but which are used in our own system design, or in other fields where similar combinations of machine learning and search are utilised, is covered. We then discuss where combinations of machine learning and search which have crossover to our intended system design have already achieved impressive results. Finally, we briefly cover IMPSY, the system our research builds on, how it combines many computer music and machine learning techniques previously discussed to great effect, what its weaknesses and gaps in function are, and how applying combined machine learning and search techniques found in other fields can address some of these weaknesses.

2.1 New Interfaces for Musical Expression (NIMEs)

The Cambridge Companion to Electronic Music by Collins and D’Escriv  n (2012) describes three eras of music programming. The first era began with MUSIC-1, the first digital computer music synthesis program, which was followed by a series of versions collectively referred to as MUSIC-N. MUSIC-N introduced the concept of algorithmic composition, and is the precursor to modern computer music. The second era was defined by the introduction of real-time music programming languages, such as Max (Puckette, 2002) and Pure Data (Puckette, 1997). Pure data is a graphical drag and drop modular programming language which allows for algorithmic compositions to be edited during performance, by modifying the code of the system while a performance is in progress. While live performance with pure data is rudimentary compared to modern alternatives, it was a large step forward in the computer music space. The third and current era of music programming refers to the surge in more real-time and interactive musical interfaces, and a shift from programming being a tool to create musical algorithms, instruments, and performances, to programs becoming the instruments themselves. NIMEs are at the core of what this current era represents. The term NIME originally comes from a conference series of the same name, which began in 2001 and now annually showcases the latest NIME related research.

The key category of NIMEs we are interested in for our research are real-time interactive NIMEs. These are NIMEs where the user and system interact in real-time during musical performance, and can be approximately split into three main subcategories. The first is NIMEs focused on the mapping from a controller to a musical output, such as gesture based controllers, or augmented instruments. The second involves simultaneous collaborative performances involving independently acting users and systems, such as human-robot collaborative ensembles. Finally, the third subcategory is interactive musical prediction systems, where the system learns from the user and contributes accompaniment, including in the form of call and response. The call and response mode of interactive musical prediction systems is the main focus area of this thesis.

2.1.1 Controller Mappings

A type of controller which mandated innovation in mapping controller inputs to musical outputs is gesture-based controllers. Early gestural musical controllers such as the Radio Baton and Conductor Program helped to establish early frameworks for three-dimensional gestural control (Lawson and Mathews, 1977; Mathews, 1991). These advances were a step toward the use of this technology in commercial devices such as the Wii Remote and Microsoft Kinect, devices which have influenced gestural controllers in many fields including music, and made gestural technology accesible and recognisable to the broader public (Kiefer et al., 2008; Zhang, 2012; Takase and Shiramatsu, 2020).

Other work focused on making mapping strategies which were generalisable to many types of input and output. Hunt et al. (2000) introduced a generalised mapping strategy which uses a two-stage rules-based concept which could allow for any input controller

2.1 New Interfaces for Musical Expression (NIMEs)

to control the same musical output function, this was done by the initial stage, which is controller specific, converting controller inputs into values of broad musical concepts, and a second stage converting those values into musical outputs. The Wekinator, created by Fiebrink et al. (2009) was even more generalisable, allowing performers to train custom machine learning models to map controller inputs to musical outputs through user provided training examples. Figure 2.1 shows this interactive learning process, which is similar to the process used in IMPSY Martin and Tørresen (2019). More recent applications involve mapping strategies where the mapping itself can perform the vast majority of the music generation, while still providing a level of control to the user, such as text to music generation models (Wu et al., 2024).

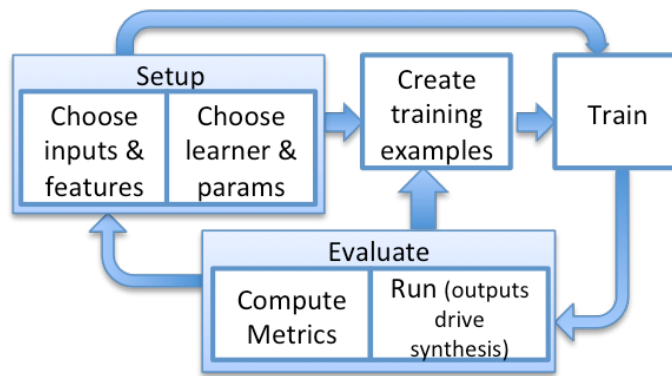


Figure 2.1: Reproduction of Figure 1 from Fiebrink et al. (2009). A flowchart of the interactive mapping generation process in Wekinator.

Augmented traditional instruments represent another significant approach, and can add enhanced control capabilities to instruments, adapt their outputs, or both, without sacrificing their familiarity. Machover and Chung (1989) created the hyperinstruments project, which was an early work focusing on control capabilities by enhancing traditional instruments with sensors and digital processing capabilities. A more recent example is the Magnetic Resonator Piano by McPherson (2010), which utilises electromagnetic actuators to directly manipulate acoustic piano strings, creating new sound possibilities while maintaining the acoustic piano as a base.

Our research aims to be agnostic to the type of controller, mapping strategy, and output used in a similar way to the Wekinator (Fiebrink et al., 2009), as any instrument which can map its controls to a set number of dimensions can be used to train a model for the system. This is particularly important for our research, as the system is designed to adapt to the unique control strategies of each performer, and it would be odd to enable this adaptation for only a single instrument.

2 Background

2.1.2 Collaborative Networked Instruments

Collaborative networked instruments have extend the concept of NIMEs beyond individual performance, allowing multiple performers to interact simultaneously across physical and virtual spaces, and for AI and human performers to collaborate in simultaneous performance. While more primitive collaborative networked performances had been conducted earlier, the spreading of high speed internet in the early 2000s is what led to it becoming a more commonly researched topic (Renaud et al., 2007). Early research often focused simply on producing some kind of engaging performance which utilised a distributed computer architecture, with the core difficulties being synchronization strategies and communication protocols, for example the Stanford Laptop Orchestra (Wang et al., 2009) conducted many such performances. Other work has focused more on systems where perception of what the other performers are doing is the main difficulty, such as a system designed by Weinberg et al. (2009) where AI systems and human performers must interact and improvise in real-time to produce coherent musical output. While our research does not work with simultaneous collaborative systems, it does still require call-and-response collaboration between one human and one AI performer, and so this work on perceiving human playing style in order to respond coherently is a core part of our research. By tracking musical parameters important to the human perceptual framework, such as rhythmic stability (Desain and Honing, 2002), our system can match these parameters when responding to the human performer and hence better follow on from human improvisation. While these methods are very common in collaborative networked systems, they are not as common in the machine learning based interactive musical prediction systems we aim to improve upon, and so integrating these methods into our system could fill a gap in that field.

2.1.3 Interactive Musical Prediction Systems

Interactive musical prediction systems anticipate and respond to performer actions based on learned patterns and musical knowledge. These systems include rule-based systems based on knowledge of musical theory, either learned or hardcoded; markov-based approaches, probability trees, and other types of probabilistic predictions of a next musical action; and various neural network architectures capable of learning to make musical predictions from scratch (Collins, 2007). An example of a markov-based system is the Continuator by Pachet (2002), which builds a database of recognised musical patterns from a training set of human improvisation, and from this database generates a markov model to generate predictions and continue from human performance in call-and-response fashion. These earlier types of call-and-response systems form the background for the development of IMPSY, the core system our research builds on (Martin and Tørresen, 2019). IMPSY uses a mixture density recurrent neural network (MDRNN) to allow for collaborative call and response performances between human performers and an AI system by predicting the most likely next instrument control values. IMPSY will be explored in more depth later in the chapter.

2.2 Machine Learning in Music

Machine learning techniques expand the possibilities of NIMEs significantly by allowing for sophisticated and human-like generative music. Unlike rule-based approaches which rely on hardcoding of musical knowledge, machine learning models can discover patterns and relationships directly from musical data, allowing for more adaptive musical interactions. A well-trained musical prediction model can anticipate likely continuations of musical phrases, suggest complementary harmonizations, or generate real-time accompaniments to human performance on a variety of instruments. NIMEs can also be generalisable to multiple models trainable by users, allowing for easy expansion to different instruments and performance styles. The creation of effective machine learning based NIMEs requires an ability to capture both local patterns and global structures inherent in music. They must also balance the advantages of purely exploitative strategies, which aim to consistently generate the most likely next note, with explorative ones, which aim for more unique and human-like results. Many different machine learning architectures have been explored in attempt to balance these objectives, and each offers their own advantages and disadvantages. This section examines five key approaches that have proven particularly valuable for musical prediction tasks.

2.2.1 Markov Models

Markov models have been widely used in algorithmic composition due to their ability to capture local musical patterns and transitions. These probabilistic models operate on the principle that future states depend only on the current state, as well as sometimes a finite history of states, making them particularly suitable for modeling sequential musical events where local context strongly influences prediction. They are incredibly efficient computationally, requiring only a transition matrix to represent the probabilities of moving between states. However, this simplicity comes at the cost of limited expressive power, as Markov models struggle to capture long-term dependencies and hierarchical structures present in music.

The most basic form of Markov model is a first-order Markov chain. These chains model transitions where the next state depends solely on the current state, represented by a simple transition probability matrix. While computationally efficient, they often produce musically simplistic results that cannot replicate long-term structure, as shown by [Zicarelli \(1987\)](#) in Figure 13. In contrast, higher-order Markov chains incorporate longer histories by conditioning predictions on sequences of previous states, enabling more coherent musical phrases and motifs. Figure 2.2 shows how a basic second-order Markov model of chord transitions can capture more complex patterns by considering the last two states, but with a larger model size.

Higher order markov chains, while more powerful, suffer from exponential growth as their order increases. This problem led to [Ron et al. \(1996\)](#) developing the Variable-order Markov model (VOMM), which adaptively determines the appropriate context length for prediction based on available training data. This adaptability allows VOMMs

2 Background

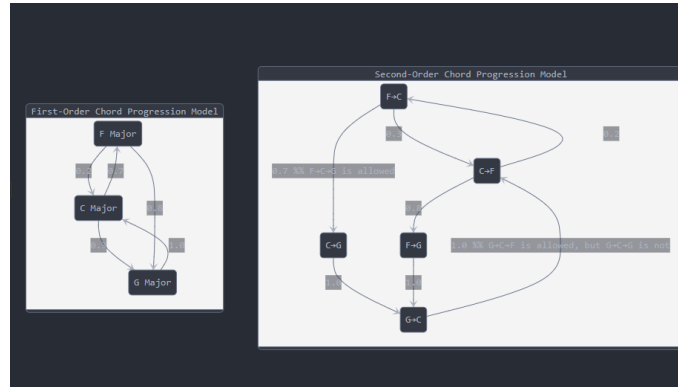


Figure 2.2: TODO: THIS IS A DRAFT MADE BY CLAUDE. A second order markov model can incorporate more complex patterns at the cost of increased size.

to capture both short patterns when data is sparse and longer sequences when following significantly well travelling paths in the training data, making them particularly valuable for interactive systems where response time and predictive accuracy must be balanced.

Hidden Markov Models (HMMs) are another type of Markov Model where the states are only partially observable, and the goal of the system is to predict the most likely series of states which generate a visible set of transitions. For example, [Allan and Williams \(2004\)](#) show how having chords as hidden states, but melody as a visible transition, can create a model which predicts viable chord accompaniments to melodies. [Simon et al. \(2008\)](#) develops this concept further, allowing users with no musical experience to sing and have accompaniment automatically generated for them by a HMM. HMMs are particularly useful in these kinds of cases where there is a clear distinction between a known element, and an unknown element which must be generated based on the known element.

Markov models effectively generate simple accompaniments, recognise common patterns in musical styles, create reasonable results with limited training data, and can easily be made computationally efficient enough for real-time applications. However, they are limited by their simplicity, as they can only capture local patterns and transitions, and struggle to model complex musical structures. This limitation has motivated the development of more sophisticated neural network architectures that can capture more complex and less well-defined aspects of music.

2.2.2 Feedforward Neural Networks

Feedforward neural networks (FNNs) are a step inbetween markov models and recurrent neural networks. Unlike Markov models, which function based on explicit transition probabilities, FNNs can discover non-linear relationships within musical data, which can capture more complex musical structures. However, FNNs face limitations in handling sequential data, as they require fixed-size inputs and lack mechanisms for capturing

long-term dependencies. The name ‘feedforward’ refers to the one directional flow of information through the network, where each layer’s output serves as the input to the next layer, without any loops or backward flowing feedback connections. Figure 2.3 shows how a feedforward network could be designed for an augmented instrument mapping strategy, where the first layer is the input layer, which could encode the controls of the instrument, there are then many hidden layers which transform these controls in complex ways, and then finally an output layer which produces a sound.

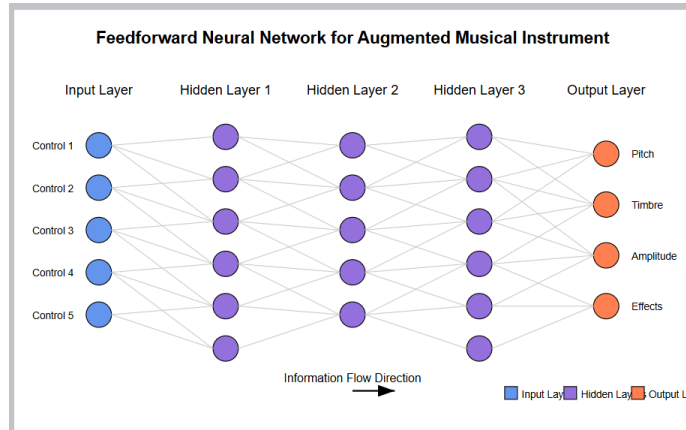


Figure 2.3: TODO: THIS IS A DRAFT MADE BY CLAUDE, actual diagram should have image of instrument at start, and output images at end. An example feedforward architecture used as a mapping for an augmented instrument.

While FNNs could theoretically be used for simple sequential musical functions, they are much better suited to musical classification tasks, such as evaluating rhythms (Dolson, 1989), chord classification (Laden and Keefe, 1989), and musical style classification (Dannenberg et al., 1997). This is because their limitations made them unsuitable for sequential tasks such as musical prediction or generation. Their requirement of a fixed size input size means that musical sequences must be encoded into fixed length vectors, often forcing dimensionality reduction which leads to the loss of important information. Additionally, their lack of feedback connections means that they cannot remember previous inputs, making them unsuitable for temporal data like music. However, FNNs established important foundations for the later development and use of recurrent neural networks.

2.2.3 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) extend beyond the limitations of both Markov models and feedforward neural networks by maintaining internal states that can capture more complex temporal dependencies in musical sequences. Unlike FNNs, which process inputs independently with no inbuilt mechanism for processing sequences, RNNs incorporate backward flowing feedback connections which allow information to persist across time steps (LeCun et al., 2015). In addition, RNNs utilise parameter sharing, which

2 Background

allows them propagate learning weights within the network over multiple time steps, and hence not have to repeatedly relearn already determined musical rules which are likely to persist over time. However, RNNs can suffer from the vanishing and exploding gradient problems, where gradient information either diminishes or grows exponentially as it is propagated backward through time (Bengio et al., 1994). In practice, this means basic RNNs often fail to capture important long-range dependencies that create coherent musical phrases and sections.

RNN architecture consists of a hidden state that acts as a form of memory, allowing information to persist across time steps. This hidden state is updated at each time step based on both new input and its previous value, making RNNs particularly suited for sequence modeling tasks. Weights are kept when the hidden state is updated, allowing the network to learn long-term dependencies in the data. Figure 2.4 shows a simple RNN architecture, where the input is fed into the network at each time step, and the hidden state is updated based on both the input and the previous hidden state. The output of the network is then generated from the hidden state, which can be used to predict the next note in a sequence. An exploding gradient happens when the weights are too large, causing the gradients to grow exponentially as they are propagated through timesteps, and vanishing gradients is the inverse problem.



Figure 2.4: TODO: THIS IS A DRAFT MADE BY CLAUDE. TODO: show vanishing/exploding gradient problem? An example RNN used for sequential note prediction.

Todd (1989) was an early example of using RNNs for composition, and split music into a series of discrete 16th note blocks, which were then used to train a simple RNN to predict the next note in the sequence. The recurrence in the network allowed it to learn melodies up to 20 notes in length, and this helped it to generate melodies with

longer dependencies than previous markov model approaches. Another example is the CONCERT system (Mozer, 1994), which includes pitch, duration, and chord as outputs. This removes the need for discrete 16th note blocks, as duration is now a continuous variable. It can also learn at the phrase level to generate higher level musical structures, and was shown to be both more concise and more general as a model than third order markov models.

RNNs established the foundation for neural sequence modeling in music generation systems and demonstrated the potential for learning-based approaches to capture musical relationships across multiple time scales. However, their architectural limitations, particularly in regards to gradient issues preventing long-term memory, ultimately motivated the development of more sophisticated recurrent architectures specifically designed to address the challenges of long-term dependency modeling.

2.2.4 Long Short Term Memory Networks

Long Short-Term Memory networks (LSTMs), created by Hochreiter and Schmidhuber (1997), are a type of RNN which addresses the vanishing and exploding gradient problems of standard RNNs, enabling more effective learning of long-term dependencies crucial for musical coherence. Their gated architecture provides selective memory capabilities that better align with musical structures spanning multiple time scales. LSTMs can remember critical information over extended periods, but also forget irrelevant information, keeping them computationally efficient and effective for music generation tasks. Most of the time that RNNs are mentioned in other sections of this thesis, we are referring to LSTMs, as these are the most commonly used RNNs in music generation tasks today (Ji et al., 2024).

The LSTM architecture introduces three specialized gates; the input, forget, and output gates; which regulate information flow through the network. These gates determine which information should be added to the cell state, which should be removed, and which should be output, allowing the network to selectively preserve important features while discarding irrelevant ones. This selective memory mechanism is particularly valuable for music, where relationships between notes can span varying temporal distances depending on the musical context. Figure 2.5 shows a simple LSTM architecture for sequential note prediction.

In music generation, LSTMs have demonstrated remarkable capabilities. Unlike standard RNNs, they can maintain coherent harmonic progressions across extended musical phrases while simultaneously managing multiple independent voices. The first notable use of LSTMs for music generation was by Eck and Schmidhuber (2002), which trained an LSTM to generate blues improvisations. It had improved structural coherence over standard RNNs, although still not close to the structural coherence of human performance. A more modern LSTM prediction system is DeepBach (Hadjeres et al., 2017), which attempted to generate convincing chorale harmonizations in the style of Bach. The results showed that on average, around 50% of listeners thought a given DeepBach

2 Background

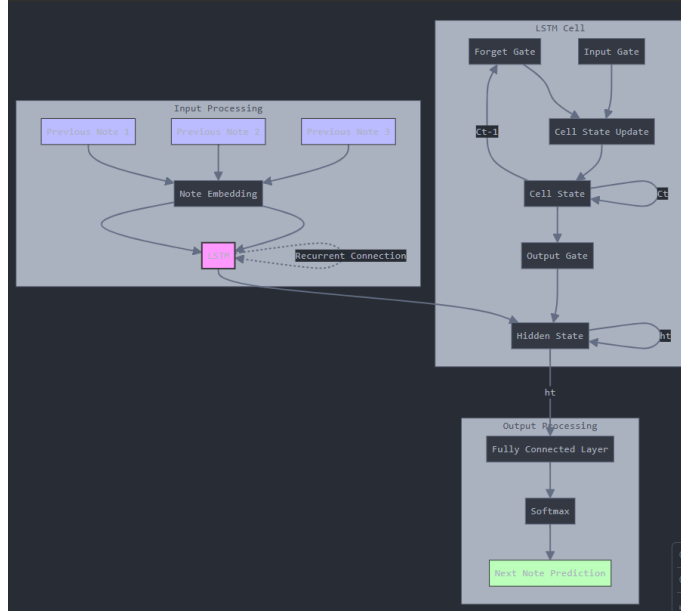


Figure 2.5: TODO: THIS IS A DRAFT MADE BY CLAUDE. An example LSTM used for sequential note prediction.

harmonization was composed by Bach, compared to scores of around 80% for Bach’s actual compositions. This shows that LSTMs can produce results which are very close to human performance, and even fool expert listeners in some cases. However, this system still had issues with understanding relative lessons about music, as it was not translation invariant, and was still only operating with relatively short pieces in a very limited and rules-based style of music. [Johnson \(2017\)](#) provided a solution to the former problem by developing a method to make translation invariant LSTMs, however, this method was only effective when trained on a single musical style, similarly to how DeepBach was, and training on multiple styles would lead to confusing stylistic shifts within a piece. The DeepJ model ([Mao et al., 2018](#)) later solved this issue by adding specific style conditions at each layer of the network.

Despite their advantages over standard RNNs and multiple decades of improvements, LSTMs still face multiple limitations ([Ji et al., 2024](#)). They still struggle with very long-term dependencies, such as musical motifs. They also struggle with more variable musical structures, such as those found in jazz or free improvisation as opposed to classical music, where the relationships between notes are less predictable. This is because partially due to LSTMs producing only a single output instead of a probability distribution over possible continuations. A solution to this latter limitation is to use a softmax layer at the output of the LSTM, which allows for a probability distribution over the possible next notes. However, this approach is limited to discrete possibilities, and so cannot capture the full range of possibilities, especially for many dimensional continuous musical parameters for instrument output.

2.2.5 Mixture Density Recurrent Neural Networks

Mixture Density Networks (MDNs) are a type of neural network which output a Gaussian Mixture Model (GMM), which is a weighted sum of multiple Gaussian distributions across any number of dimensions. This GMM is used as a probability distribution for the output note, as opposed to traditional networks, which output only a single output option (Bishop, 1994). Mixture Density Recurrent Neural Networks (MDRNNs) are an RNN with a GMM output, and can solve the issue of RNN methods being seen as producing results which are too robotic or predictable over the long term to be truly human-like by allowing for the benefits of softmax outputs to be extended to continuous musical parameters. It also extends recurrent architectures in a way which allows them to better handle multiple continuous dimensions simultaneously, such as pitch, time, timbre, and many more, enabling more holistic prediction models.

MDRNNs incorporate a GMM output by outputting the necessary parameters vectors π , μ , and σ , each of which the same dimension as a standard single output. The π parameter is the weight of each Gaussian in the mixture, the μ parameter is the mean of each Gaussian, and the σ parameter is the standard deviation of each Gaussian. The output of the MDRNN is then a sample from the GMM, which can be used as the next note in the sequence. Figure 2.6 shows a simple MDRNN architecture for sequential note prediction. The output of the MDRNN are the inputs to a GMM, which is then used to sample the next note in the sequence.

MDRNNs have been used in multiple human prediction domains, including generating handwriting (Graves (2013)), sketching (Ha and Eck (2017)), and touchscreen performances (Martin and Tørresen, 2018). In all cases GMMs were used to generate more flexible outputs in continuous domains where the goal was to seem human-like, and performed better than softmax outputs. Hence, MDRNNs are a natural fit for music prediction systems, where this is also the goal, and have succeeded in this domain previously too (Martin and Tørresen, 2019). This is because MDRNNs can probabilistically express future likelihoods for continuous parameters such as timing, dynamics, timbre, and articulation; creating natural variations in expression that more closely resemble human musical composition.

However, the benefits MDRNNs to musical prediction tasks come with a caveat. While the addition of probabilistic outputs introduces useful variability, this approach often amounts to merely adding noise to otherwise deterministic predictions. This is in contrast to human improvisation, which introduces variations with intentional structural and semantic meaning rather than random deviations. In this sense, MDRNNs take the disadvantages of standard RNN outputs being too structured, and move too far in the opposite direction, instead producing outputs with arbitrary randomness. Ideally, a system would be able to balance these two extremes, producing outputs which are both structured and meaningful, but also varied and creative. This is exactly what our research aims to achieve, by replacing the final step of note generation in a standard MDRNN, random selection from the generated GMM, with an algorithm which

2 Background

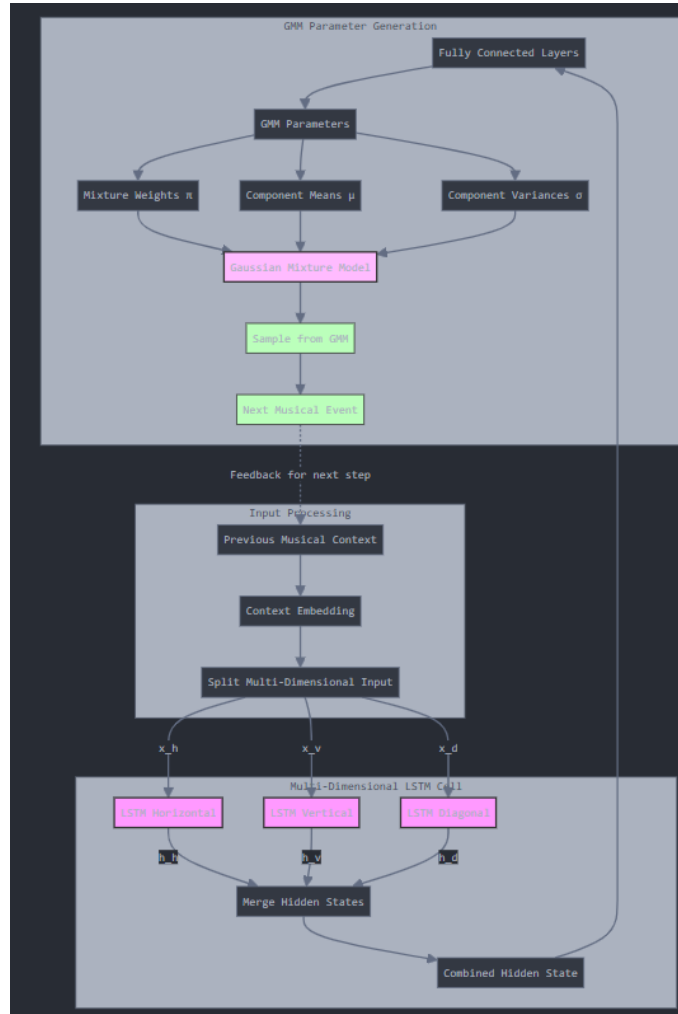


Figure 2.6: TODO: THIS IS A DRAFT MADE BY CLAUDE. An example MDRNN used for sequential note prediction.

meaningfully guides the selection process.

2.3 Heuristic-Based AI in Music

Heuristic methods serve as an alternative to machine learning based approaches for AI music generation and prediction. While neural networks excel at capturing the statistical patterns in music, they often lack the structured knowledge that can be encoded through rules and search algorithms. Heuristic approaches provide a framework that can enforce musical constraints, guide exploration of the solution space, and incorporate domain knowledge that may be difficult to learn from data alone. This section examines two approaches to heuristic-based AI in music: rule-based systems, and search algorithms.

The search algorithms subsection focuses on hybrid approaches that combine heuristics with machine learning models.

2.3.1 Rules Based Systems

Rule-based approaches have been used in algorithmic composition since the 1950s ([Hiller and Isaacson, 1959](#)), providing well-defined frameworks for generating music according to hardcoded constraints. These systems typically encode music theory knowledge, stylistic conventions, and compositional techniques into these constraints, which then guide note selection and musical structure. This allows for the generation of music that is very structured and coherent, at the cost of creativity, flexibility, and adaptability to novel musical contexts.

The earliest known example of a computer generated composition was the ILLIAC Suite ([Hiller and Isaacson, 1959](#)), which, while technically a markov model based system, used the markov model only for adding randomness to an otherwise completely rule based system. Another technique for rule-based composition is the use of grammars, which define a set of rules for recursively generating musical structures. [Steedman \(1984\)](#) used such a grammar to generate jazz chord progressions, demonstrating how formal grammars can capture the underlying structure of musical styles. Rules based systems like these almost always focused on a single style, and were the most prevalent method of computer music generation in its early years ([Moorer, 1972](#)).

Sometimes rule-based systems utilise constraint satisfaction programming (CSP), a hybrid between rule-based and search-based systems for very complex sets of musical rules, where the only objective of search is to find a solution which adheres to rules, which are encoded as constraints in a CSP algorithm. This is a declarative method, where only the rules are defined and then the system itself decides how to apply them. This allows CSPs to find solutions where multiple competing constraints, such as voice leading and harmonisation, exist. An example of this approach is the CHORAL system, intended to harmonise four-part chorales in the style of Bach by combining a set of over 270 musical rules ([Ebcioğlu, 1988](#)). [Anders and Miranda \(2011\)](#) survey extensive research into both the creation of useful constraints for music, and their use in general systems, most of which employ CSPs.

In the context of NIMEs, rule-based systems offer several advantages: they provide predictable, theory-compliant outputs; they can incorporate domain-specific knowledge about particular instruments or styles; and they can generate results in real-time with computation time dependant only on the complexity of the ruleset. However, pure rule-based systems face significant limitations. They struggle with the flexibility and nuance of human performance, often producing robotic results that lack the subtle variations of true expressive composition. Additionally, the manual encoding of musical knowledge becomes infeasible when attempting to capture the totality of musical composition across musical styles, and instead extensive hand-crafting of separate rule sets is usually required for different genres or instruments. This leads to a lack of adaptability to novel

2 Background

musical contexts not specifically encoded into the rule system.

2.3.2 Search Based Systems

Search algorithms combined with musical heuristics enable more sophisticated composition by exploring possible continuations and selecting optimal musical paths according to defined criteria. These approaches view music generation as a search problem, where the varying array of possible musical continuations is an important component of composition, and pleasing musical composition comes from exploring the space of possible continuations and selecting the most promising ones through the use of effective heuristics.

The use of tree search algorithms as the sole process of music generation is rare, as typically either constraint based search methods such as CSPs will be used, or search algorithms will be used in combination with machine learning models. Hence, the discussed papers all combine tree search with some form of machine learning architecture. Such hybrid approaches represent a promising direction for NIMEs, leveraging the pattern-recognition capabilities of neural networks while enforcing structural coherence through rules and search strategies. These systems typically employ neural networks for generating basic musical material but constrain or guide these generations using rule-based systems. Our research is a hybrid approach of this type, and so research into these systems is particularly relevant to our work.

[Fu et al. \(2023\)](#) developed one of the few examples of a Monte Carlo Tree Search (MCTS) based musical prediction system, using a transformer model as a successor function to explore the space of possible continuations for a monophonic instrument, and selecting the most promising ones using heuristics founded on rule-based methods. This approach allows for the generation of music that is both structured and creative, as it can explore a wide range of possibilities while still adhering to musical constraints. The results showed high coherence, and better accuracy at making musical predictions than either purely rule-based or purely machine learning based methods. This demonstrates the potential of our research, as we are also aiming to utilise MCTS to increase the coherence and predictive accuracy of a machine learning based system. While this paper is a promising example of the use of MCTS for guiding music generation, it doesn't include another key aspect of our research, which is the detection and continuation of key musical parameters in a piece through MCTS guidance. However, this use case of MCTS in music generation has been explored by [Ferreira et al. \(2022\)](#), who used MCTS to steer a neural language model toward a particular desired emotion, and showed very promising results compared to other methods in human listening trials. Despite its specificity to a single musical parameter, this work demonstrates the potential of MCTS for guiding the generation of music toward specific goals, and in combination with the research of [Fu et al. \(2023\)](#), provides promising background for the effectiveness of techniques used in this thesis.

The effectiveness of hybrid approaches underscores a key insight for NIME development: rather than viewing heuristic-based and learning-based methods as competing

approaches, they can be integrated to create systems that leverage the strengths of both. However, there is a gap in the application of hybrid approaches, as research into the guiding MDRNN based search using MCTS with heuristics based on desired musical parameters does not exist, despite all three components having been tested with positive results in the literature (Martin and Tørresen, 2019; Fu et al., 2023; Ferreira et al., 2022). This is the gap that our research aims to fill, as MDRNNs could benefit from the structured guidance of heuristic search algorithms similarly to how other approaches have in this related research.

2.4 Tree Search Algorithms

Our research focuses on the ability of tree search algorithms, in particular Continuous Monte Carlo Tree Search (C-MCTS), to guide the selection process from a GMM output by an MDRNN in an interactive musical prediction system. This requires an understanding of C-MCTS, and the algorithms it is built upon, Beam Search and Monte Carlo Tree Search (MCTS). This section will provide a theoretical overview of these algorithms, and some of their early applications in fields outside of music, given that their use within music is limited, to provide the background necessary for understanding the technical aspects of this thesis. It will also provide a brief overview of why they are applicable to our research goals.

2.4.1 Beam Search

Beam Search offers a computationally efficient alternative to more exhaustive search methods by maintaining only the most promising partial solutions at each step of sequence generation. It can be thought of as a compromise between greedy search, which selects only the current most optimal candidate at each step, and exhaustive search methods such as Breadth First Search (BFS), which explore all possible branches to a given depth. This is because beam search retains the top- k candidate sequences based on cumulative heuristic scores at each step, extending greedy search while maintaining computational efficiency (Bisiani, 1992).

The algorithm operates by expanding all possible next steps from the current set of partial sequences, scoring each expanded sequence using heuristic evaluations, then pruning all but the k -highest scoring sequences. This process continues iteratively until complete sequences are generated or a termination condition is met. Figure 2.7 shows an example of a beam search with $k = 2$, where the algorithm explores multiple branches of a monophonic musical prediction tree.

An early use of beam search was the Harpy system created by Lowerre (1976) for speech recognition, which maintained a set of hypotheses for possible phonetic sequences in a tree structure, and kept to a limited number of hypotheses by using beam search. Beam search continued to be used in speech recognition systems for many years, proving its effectiveness in this domain Steinbiss et al. (1994). A similar method of multiple

2 Background

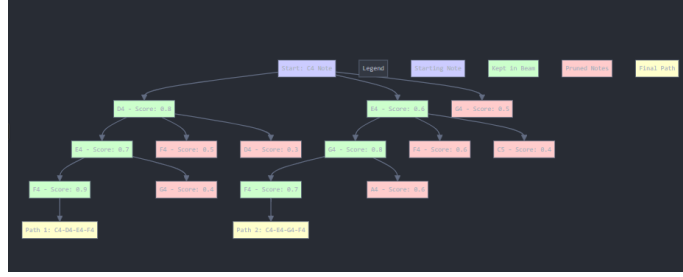


Figure 2.7: TODO: THIS IS A DRAFT MADE BY CLAUDE. An example of a $k = 2$ beam search for sequential note prediction.

hypothesis maintenance was used effectively by [Sutskever et al. \(2014\)](#) for sequence-to-sequence tasks, such as language translation, showing how a combination of beam search and neural networks can be effective.

The primary advantages of Beam Search include its relative computational efficiency and straightforward implementation. However, its limitations become apparent in generation tasks requiring exploration of diverse possibilities or long-term planning. Because Beam Search maintains only a limited set of candidates based on immediate scores, it can suffer from horizon effects where initially promising paths lead to suboptimal long-term outcomes, similarly to greedy search. In short, it mixes the advantages and disadvantages of greedy and exhaustive search methods.

2.4.2 Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) addresses the limitations of Beam Search by balancing exploration and exploitation through a combination of tree search and random sampling. MCTS constructs a search tree incrementally through repeated simulation, building confidence in promising paths over time through selection, expansion, simulation, and backpropagation. This process is explained below, and can be seen used in Figure 2.8 for a sequential note prediction task.

1. **Selection:** Starting from the root node, the algorithm traverses the existing tree by selecting child nodes according to a tree policy, typically using the Upper Confidence Bound for Trees (UCT) formula (Kocsis and Szepesvári, 2006) (TODO: maybe change this formula to use expected reward + c times sqrt formula if that is what we use for system design, could be so if we use probability as expected reward):

$$\text{UCT}(n) = \frac{Q(n)}{N(n)} + c \cdot \sqrt{\frac{\ln(N(p))}{N(n)}} \quad (2.1)$$

where $Q(n)$ is the accumulated reward at node n , $N(n)$ is the visit count, $N(p)$

is the parent’s visit count, and c is an exploration constant. The UCT formula balances exploitation (favoring nodes with high rewards) and exploration (favoring nodes with low visit counts), and this balance can be tweaked by adjusting the exploration constant.

2. **Expansion:** Upon reaching a leaf node, the tree is expanded by adding one or more child nodes representing possible next actions.
3. **Simulation:** From the newly added node(s), a simulation is conducted to estimate the value of the node, typically by sampling actions according to a simple default policy involving some degree of randomness until reaching a terminal state. In simulations involving very large depth, the simulation can be cut off at a certain depth and the value of the node can be estimated by a heuristic.
4. **Backpropagation:** The simulation result is propagated back up the tree, updating visit counts and heuristic values for all nodes along the path.

While monte carlo methods, where random sampling is used to simulate outcomes to deterministic problems, were first developed by [Metropolis and Ulam \(1949\)](#) in the 1940s and have been used in search algorithms for many years [Brügmann \(1993\)](#), the first use of MCTS in its modern form was by [Coulom \(2006\)](#), who used it to develop Crazy Stone, a Go playing AI. The UCT selection formula was also only developed in 2006 ([Kocsis and Szepesvári, 2006](#)), and so it is only from this point that the effectiveness of MCTS has become widely recognised. [Coulom \(2006\)](#) demonstrated that MCTS could significantly outperform MiniMax search in games with large branching factors, such as Go, by effectively balancing exploration and exploitation. The algorithm’s ability to learn from random simulations allowed it to adaptively focus on promising regions of the search space, leading to improved performance in complex decision-making tasks. MCTS was very quickly adopted into a variety of domains, including game playing, sample-based planning, and optimization problems, where these vast search spaces are common ([Browne et al., 2012](#)). More recently, MCTS has also seen a lot of success when combined with deep learning, and such hybrid cases will be discussed in the next section.

A significant advantage of MCTS over Beam Search is its ability to balance exploration of the search space with exploitation of promising paths in a more natural way. Instead of maintaining a fixed set of candidates, MCTS adapts its search strategy to focus more on promising regions of the search space as it accumulates more information. Through repeated simulation, MCTS gradually builds confidence about the value of different choices, enabling more sophisticated long-term planning. However, standard MCTS implementations face challenges in continuous domains, where the state and action spaces cannot be easily discretized.

2.4.3 Continuous Monte Carlo Tree Search

Continuous Monte Carlo Tree Search (C-MCTS) extends traditional MCTS to handle continuous state and action spaces, making it particularly suitable for exploring contin-

2 Background

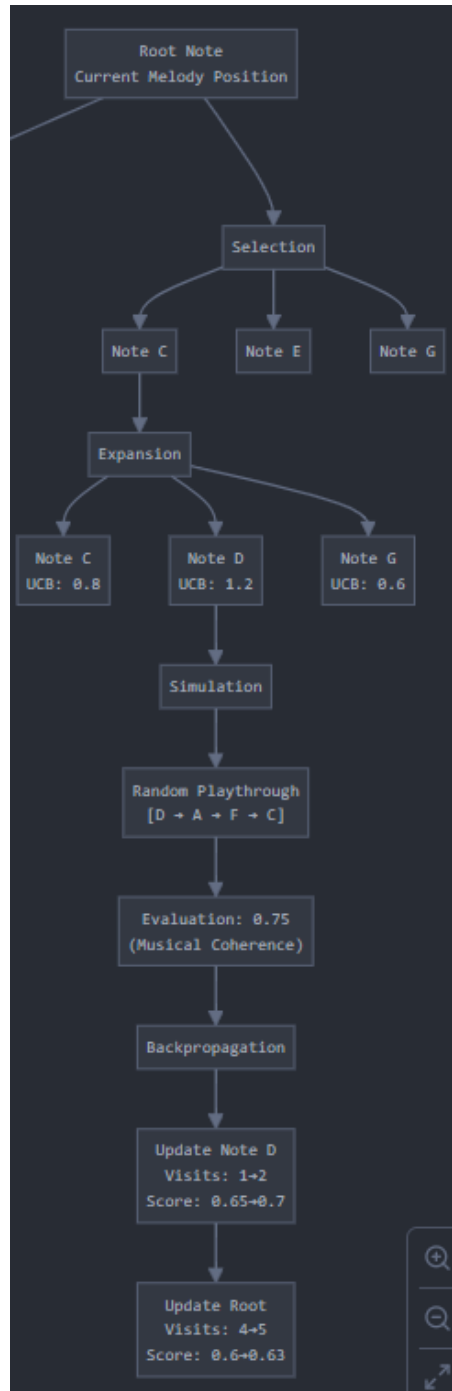


Figure 2.8: TODO: THIS IS A DRAFT MADE BY CLAUDE. An example of MCTS search for sequential note prediction.

uous domains, such as GMM outputs we aim to use it to explore in this thesis. Several approaches have been developed to handle continuous spaces in MCTS (Lei et al., 2021), but we will focus on the most relevant to our research.

Progressive Widening addresses the challenge of infinite branching factors in continuous spaces by gradually increasing the number of child nodes as the parent node receives more visits. Formally, if a node has been visited n times, the number of allowed children is limited to kn^α , where k and α are tuning parameters. When new children are not permitted, standard UCT selection is used to select a child node. This allows for promising nodes to be explored sufficiently before adding new children. When a new child node can be generated, it can either be randomly sampled from the probability distribution of the action space, for example the GMM output of an MDRNN in our case, or can use a probability distribution influenced by potential, where potential can be estimate using kernal-based methods.

Kernel-based methods use kernel functions to generalize rewards across similar states and actions in continuous spaces. Rather than treating each continuous value as entirely distinct, kernel methods establish similarity metrics that allow information sharing between nearby points in the parameter space. This provides an advantage over standard progressive widening as it doesn't simply reduce continuous domains back down into discrete search, but instead maintains continuous domains with continuous heuristic values applied as a function across the continuous space of actions. However, this comes at the cost of increased computational complexity. Kernal Regression Upper Confidence Trees (KR-UCT) and their use in MCTS selection are discussed by Yee et al. (2016).

Baysian optimisation methods take the techniques of kernal-based methods a step further. Instead of only using a kernal function to smooth nearby values in parameter space, baysian optimisation can account for entire GMMs, providing estimated values for the entire continuous parameter space based on sampled values. This allows for a more accurate estimate of the value of a given action, but at the cost of additional computational complexity. It has been primarily implemented for solving partially observable Markov decision processes (POMDPs) (Mern et al., 2021). Since our implementation of C-MCTS for structured MDRNN predictions utilises GMM outputs, and will only use limited sampling compared to many other MCTS problems, progressive widening with baysian optimisation is the most applicable method to our problem. TODO: in future, reference a figure in our system design section, as this will definitely have a figure displaying our use of baysian optimisation in C-MCTS. Get feedback from Charles over whether this figure is best placed here and referenced in our system design, or in our system design and referenced here (it seems strange to reference a Figure so far in the future of the thesis, maybe we use two figures?)

2.5 Applications of Combined Machine Learning and Search

The integration of machine learning prediction models with search algorithms has emerged as a powerful AI method across multiple domains. By combining the pattern recognition capabilities of neural networks with the structured exploration of search algorithms, these hybrid approaches achieve results that surpass what either technique could accomplish independently. These examples demonstrate how the combination of prediction and search has been successful in other domains, providing valuable insights into how these successes could be applied to NIMes.

2.5.1 Game Playing AI

Game playing systems have demonstrated perhaps the most obvious successes of combining neural evaluation with tree search methods, achieving superhuman performance in games once thought to be beyond computational reach. AlphaGo, developed by [Silver et al. \(2016\)](#), combined deep convolutional neural networks for position evaluation with Monte Carlo Tree Search for action selection to defeat Lee Sedol, considered one of the greatest ever Go players, in a five game match. This hybrid approach was further refined in AlphaGo Zero ([Silver et al., 2017](#)), which learned solely through self-play, eliminated human knowledge entirely and demonstrating even stronger performance, and then AlphaZero ([Silver et al., 2018](#)), which additionally boasted very strong results in both Chess and Shogi. Monte-carlo methods were later also used to develop Pluribus, the first superhuman level six-player texas-holdem poker AI ([Brown and Sandholm, 2019](#)). In these systems, MCTS was used primarily as a policy improvement operator, boosting the performance of an already strong neural network by looking deeply into the consequences of its predictions, or to learn from self-play. While we don't use MCTS for policy improvement in the same way, the goal of our research is still to use MCTS to improve the performance of a neural network, and so the success of these systems provides strong evidence for the potential effectiveness of this approach.

2.5.2 Natural Language Processing

Natural language processing systems have increasingly adopted hybrid approaches that combine the human imitation of neural language models with the structured guarantees provided by search and grammar constraints. Neural language models excel at capturing patterns in language, but may generate outputs that violate syntactic or semantic constraints. Search algorithms guided by these neural models can explore the space of possible outputs while ensuring adherence to formal rules. Beam search has become the main decoding strategy for neural machine translation and text generation systems [Sutskever et al. \(2014\)](#). In Google's Neural Machine Translation system, sophisticated beam search with length normalization and coverage penalties was a significant factor in the quality of their results ([Wu et al., 2016](#)). [Hokamp and Liu \(2017\)](#) extended this approach with grid beam search, which enforces lexical constraints while generating text. The success of hybrid NLP systems suggests that similar techniques could effectively

balance learned patterns with formal structure in the musical domain.

2.5.3 Robot Motion Planning

Robot motion planning and autonomous driving systems face particularly stringent real-time constraints while requiring both the adaptability of learned models and the guarantees of structured search algorithms. For motion planning problems, regular search techniques such as A* cannot generate effective paths, as during the execution of the path the environment around the agent will be dynamically changing in an unpredictable way, which can affect the validity and optimality of generated paths. Hence, because the states after executing an action are non-deterministic, RNN are used to generate predicted nodes for each point in the search tree. These RNNs are good at accounting for the complex dynamic environment, and can provide reasonable predictions for what some realistic future scenarios could be after executing a set of actions. The agent can then search through these RNN created environmental prediction trees, often using MCTS, to find paths which more effectively account for the unpredictable environment. This is a similar process to the algorithm in our research.

One hybrid method used in this space is to use an RNN to convert a non-deterministic partially-observable problem into a more simple deterministic problem which can be solved by a standard planner. [Karkus et al. \(2017\)](#) developed QMDP-net, which embeds planning within a neural architecture, integrating perception mechanisms with planning to allow for planning algorithms to better match the true environment. An inverse methodology is to reduce the action and state space to discrete actions for performing a search or planning algorithm, using a neural network at the end of simulation nodes to evaluate states. This approach is similar to AlphaGo ([Silver et al., 2016](#)), and is used effectively for motion planning by [Weingertner et al. \(2020\)](#).

The most relevant approach to our work is where the neural network is used to generate action probability distributions, either of the agent itself, of other agents, or of the environment, and then MCTS is used to search through this generated action space. One example of this is the deep-MCTS algorithm by [Chen et al. \(2020\)](#), which generated control commands for autonomous vehicles by using a combination of two deep neural networks and MCTS. The first neural network was the vehicle state prediction network, which predicted the future state of the vehicle given a current state and performed action. The second neural network was the action value prediction network, which for all possible actions provides action probabilities and expected rewards. This latter network guides the selection and expansion of nodes in the MCTS tree, which then searches predicted positive actions over multiple time steps to determine which action to take. While this approach is already very similar to our own, the KB-Tree algorithm by [Lei et al. \(2021\)](#) is even more similar, as it uses an MDRNN to generate a GMM as an action probability distribution, and then uses C-MCTS with progressive widening, updated using both kernel regression and bayesian optimisation, to search through the generated action space. KB-Tree outperformed the state-of-the-art in both reinforcement learning and MCTS-based techniques in simulated parking tests.

2 Background

The similarity of these mentioned approaches to our own research is notable, as while the domains are entirely different, the techniques used are very similar and the results are very promising. Given that these approaches are applicable to interactive musical prediction systems in a similar way, as this domain faces similar challenges to motion planning, we feel that the lack of research into applying these methods to musical prediction is a significant gap in the literature worth exploring.

2.6 IMPSY

While our research pulls from a diverse range of background knowledge, as has been explored in previous subsections, it directly builds upon IMPSY, developed by [Martin and Tørresen \(2019\)](#). Hence, an understanding of IMPSY, including its design purpose, strengths, and weaknesses, is critical to understanding the motivation behind our research.

IMPSY is a real-time interactive NIME that uses an MDRNN based approach to generate musical predictions. It is designed for collaboration with human improvisation, and individual models for various instruments and performer styles can be trained quickly by feeding the training model data from human performer improvisation on the instrument of choice. Specifically, the model is trained to predict inputs to the instrument, not its outputs, meaning that it predicts the values of instrument controls rather than directly predicting their output. This provides the benefit of the model being agnostic to the mapping strategy from instrument controls to musical output. IMPSY also has multiple different collaborative modes, such as a battle mode where a human user and the MDRNN play simultaneously, but the interaction mode we will focus on within this thesis is the call and response mode. In this mode, the MDRNN is conditioned while the user plays, and begins to generate notes itself only when the user stops, letting the user take over again whenever they begin to use the instrument. The use of an MDRNN within IMPSY solves the problem traditional RNNs of only being able to output a single prediction. By outputting a probability distribution for next musical parameter values, randomness is added into predictions, reducing the overly rigid and robotic playing style of many traditional RNNs. However, this randomness isn't a perfect solution, as the randomness is arbitrary. Humans don't just add pure gaussian randomness when improvising, such as the type that MDRNNs generate. Instead they add controlled, guided, and meaningful randomness, which balances exploration of new ideas to create a unique performance, with exploitation of known musical concepts to ensure the performance is coherent and structured.

This thesis is an attempt to more closely replicate this human improvisation process. While MDRNNs have achieved great progress in the field by combining the exploitative advantages of RNNs with the exploratory advantages of MDNs, these MDNs are unguided in their exploration. Research into other fields, particularly motion planning, has shown that guiding neural network predictions with search algorithms can lead to stronger outputs. Hence, we aim to combine the randomness of MDNs with the struc-

System Design

In this chapter we introduce the main content of our research. We begin by providing a broad overview of the design of our system, before discussing in more depth each of the components of the system which we have developed or improved, including details on their implementation.

3.1 Design Overview

TODO

3.2 Monte-Carlo Tree Search

TODO: we need a figure somewhere showing our selection algorithm (progressive widening with bayesian optimisation)

The core component of our system is the use of search to generate predictions. For this, a Monte Carlo Tree Search (MCTS) algorithm is used as the basis for generating and searching these prediction trees. MCTS is a heuristic search algorithm for decision processes, most notably employed in game playing and planning.

MCTS builds a search tree incrementally and asymmetrically, focusing on the most promising parts of the search space. The algorithm consists of four main steps: selection, expansion, simulation, and backpropagation.

- **Selection:** Starting from the root node, the algorithm selects successive child nodes until a leaf node is reached. The selection is based on a policy that balances exploration and exploitation, typically using the Upper Confidence Bound for Trees (UCT) formula.

3 System Design

- **Expansion:** If the leaf node is not a terminal state, one or more child nodes are generated and added to the tree.
- **Simulation:** A simulation (or playout) is run from the newly added node to a terminal state, using a default policy to make decisions.
- **Backpropagation:** The result of the simulation is propagated back through the tree, updating the nodes' statistics.

(TODO a more formal algorithmic version, with a diagram, of how MCTS works)

By iterating these steps, MCTS gradually builds a search tree and improves its predictions. The algorithm's effectiveness comes from its ability to focus computational resources on the most promising parts of the search space, making it well-suited for complex decision-making tasks. Additionally, as shown in the related work section, MCTS has been successfully applied in combination with MDNs in other domains, particularly robot navigation problems such as self driving cars, and has seen success. To apply MCTS to the prediction system of IMPSY, some notable challenges need to be addressed and accounted for.

Firstly, the search space of a musical piece does not have a terminal state, as unlike a game with a clear win or loss condition, a musical piece can, and in the case of IMPSY is assumed to, continue indefinitely. This means that the simulation step of MCTS cannot be run to a terminal state, and instead must be run for a set number of steps. In a basic implementation this can be a fixed number of steps, however for our system we allow the depth of search to be variable based on the 1. how promising this branch of the search tree is, and 2. how much the heuristic value appears to change with increased depth. This allows the system to focus computational resources on the most promising parts of the search space, and to avoid wasting resources on branches that are unlikely to yield useful improvements to predictions. (TODO: chuck in the more complex detail after this is coded properly)

Secondly, the search space of a musical piece is not a discrete space, but a continuous one. This means that we cannot use the traditional action selection method of MCTS, and must instead rely on a continuous MCTS algorithm.

3.2.1 Continuous Monte-Carlo Tree Search

TODO: the main function of this section should include explaining why progressive widening was chosen (we don't need the complexity of kernal based methods, as some small degree of arbitrary randomness remaining in prediction, just with added guidance, is perfectly fine. We aren't aiming to completely eliminate any element of randomness, as using only randomness has already been shown to produce good results in MDRNN NIME papers) For our Continuous Monte-Carlo Tree Search (C-MCTS) implementation, we use progressive widening. Progressive widening gradually adds children to nodes as they are visited more often, allowing the search tree to grow in a controlled manner. In particular, our version of progressive widening selects from a modified GMM produces

as an MDN output, and if the result is more than ϵ away from any existing child nodes, a new child node is added, otherwise the existing child node with the closest value is selected.

We have chosen this technique because progressive widening is simple in that it maintains a discrete search tree. Mixture Density Network outputs were originally used in IMPSY because of the benefit that a small amount of randomness provides in making the system sound more human and natural in its improvisation, and by rounding similar MDN outputs into the same discrete action in the search tree, we can maintain the benefits of this randomness, whilst also controlling it in a way which allows for musical structure.

When using C-MCTS, instead of taking a single MDRNN prediction at each step and choosing randomly from the resulting distribution, the C-MCTS algorithm generates a tree of possible future musical directions for the piece, and hence allows for heuristics to be applied in selecting the most promising direction to take according to set structural parameters. This allows for the system to make more informed decisions about the future of the piece, and to generate more structured and coherent improvisations.

TODO: explain ϵ and how it is used to determine whether to create a new successor, and how it is progressively lowered around a node whenever that node is re-explored.

3.3 The Successor Algorithm

Note how the successor algorithm is entered, eg create prediction tree at first human- $\hat{z}_t$ system prediction, continuously expand upon this tree as more system- $\hat{z}_t$ system predictions take place. Explain the algorithm used to find z_{t+1} given z_t . This includes all of the MDRNN stuff to generate a GMM, this GMM is the 'action space' for a state, use 'action space' to generate an C-MCTS tree from a z_t state. Then select the best branch via a heuristic, and hence the best z_{t+1} . Then save the tree from z_{t+1} for the next iteration of the algorithm. This is the core of the system.

3.4 Heuristics

For the C-MCTS algorithm to be useful, it must be guided by heuristics which provide useful benefit to the output of the system. In the case of IMPSY, we have developed a set of structural parameters which can be used to guide the system in generating more structured and coherent improvisations. These structural heuristics are based on the principles of music theory, and are intended to help describe the current playing style of the human performer and the current state of the piece, and hence guide the system in generating more appropriate predictions which coherently follow the current state of the piece.

Note here the limitation of heuristics needing to be defined independently for different instruments, with a few core exceptions such as purely timing based heuristics like tempo. This is because the same parameter can have different meanings for different instruments,

3 System Design

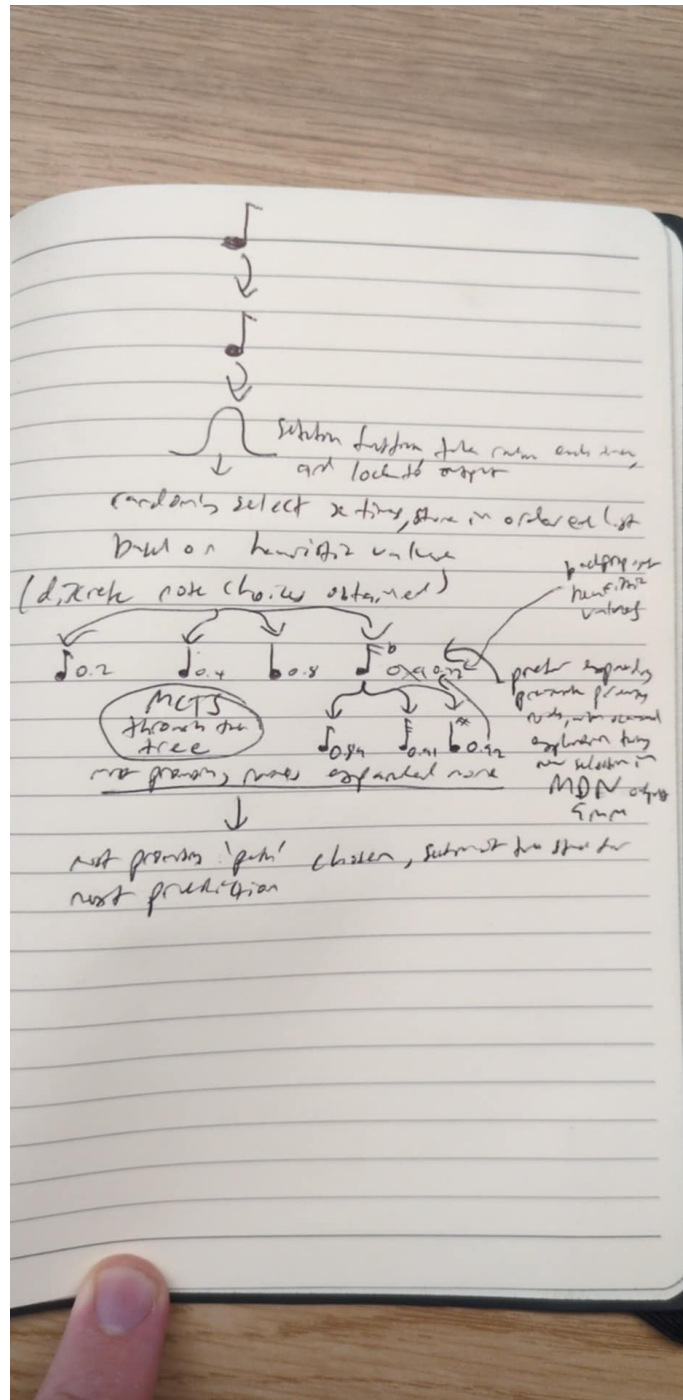


Figure 3.1: C-MCTS algorithm applied to IMPSY.

eg pitch variation for a drum kit vs a guitar. And some instruments have more dimensions

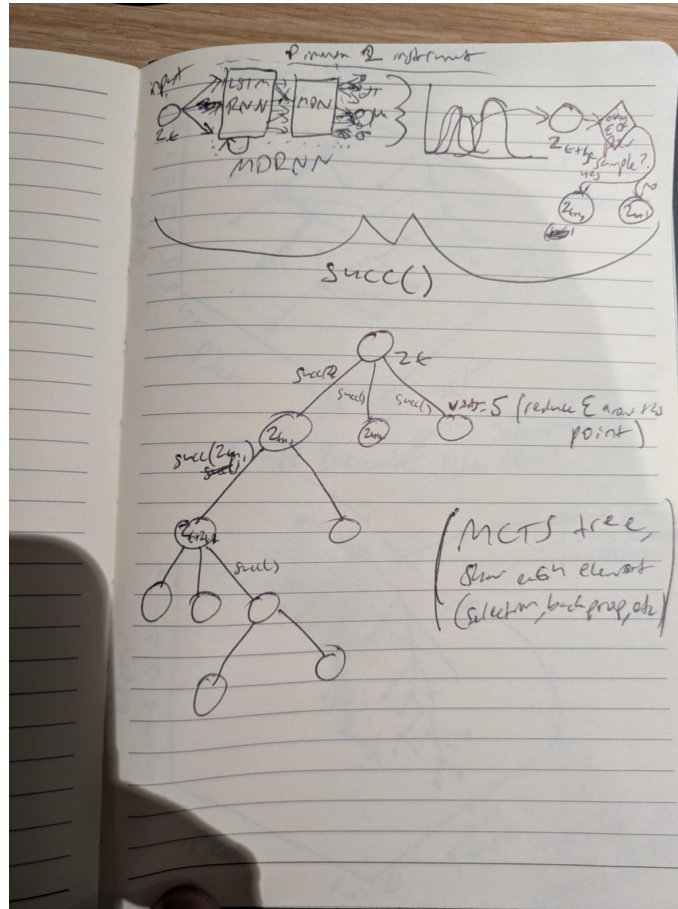


Figure 3.2: The Successor Algorithm.

in their output than others, which for each instrument can control various different things. Introduce the standard we will be using for the rest of the paper, which is a 2-dimensional keyboard output: with one pitch dimension and one time dimension. The heuristics should hence be translatable to any instrument with clear continuous pitch and time dimensions to their output, but instruments with higher dimensionality would benefit from adding more, and more complex, heuristics which account for these extra dimensions.

We want to put any general graphics pertaining to heuristics more generally here. We could (maybe should) actually do this in the main section for performance parameters before the subsections start, taking a super basic parameter like how close each time interval is to an exact figure. Below is a 3D tree graph visually displaying an example of continuous MCTS search with 1. no guidance (wide branching tree) 2. Addition of a tempo heuristic or other more basic heuristic (tree branches narrow and focus on a single direction matching the tempo) hand drawn example of this graph below. TODO:

3 System Design

in writeup make this paragraph an explanation of a specific example scenario before showing that in the figure

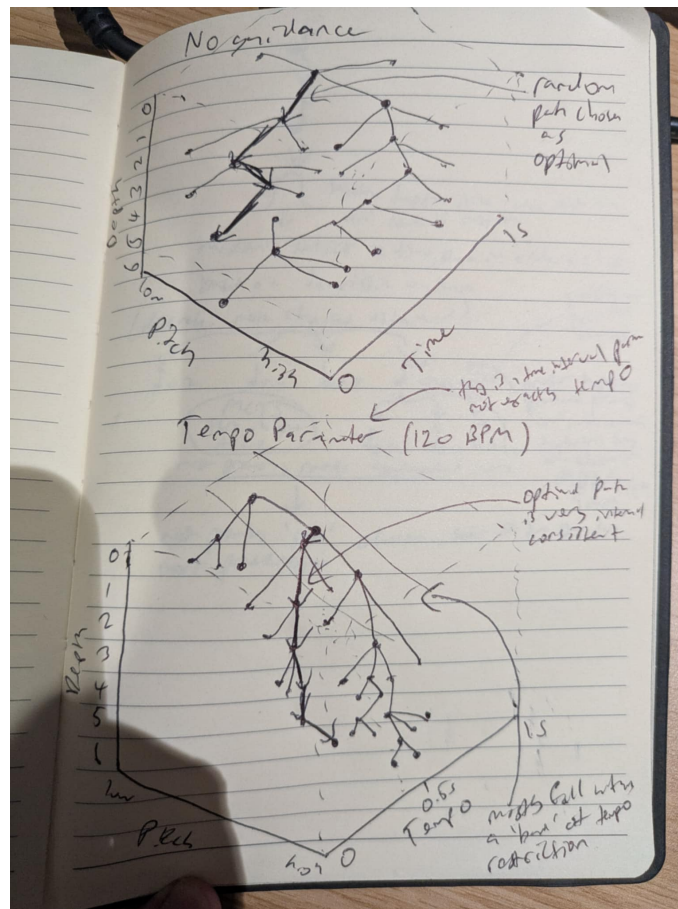


Figure 3.3: C-MCTS algorithm with no heuristic guidance vs time interval heuristic.

3.4.1 Setting Heuristics Through Human Performance

By default, for each heuristic, the system tracks both the value of the heuristic, and the rate of change of the heuristic, being its derivative with respect to time. Higher order derivatives can also be added to track more complex patterns of change in these values. In this way, the system tracks useful metrics about the the state of the piece, and then can replicate these patterns when it takes over in its improvisation.

TODO: We could just do this with another LSTM trained on the same training data? Use LSTM to estimate values of parameters for next step, and then use these as the heuristic?

TODO figure

3.4.2 Identifying Patterns of Repetition

Derivatives aren't enough to capture the change of musical parameters in improvisation. Repetition and motifs are critical to musical performance, and recognising and being able to repeat them is a core component of strong improvisational coherence. TODO: we need to figure out a way to identifying repeated patterns so that this can be used to generate a 'future function' of target values for parameters. We also need to ensure we don't forget that we are in a motif. Eg we strongly need to preference continuing a motif if we have begun an identified motif.

TODO figure

3.4.3 Utilising Heuristics for Prediction

When the system takes over from a human performer in an improvisation, it forecasts the future values of each structural parameter based on 1. the current value of the parameter, 2. derivatives of the parameter, and 3. identified repeating pattern of the parameter. A heuristic can then be created from the forecasted values, where the heuristic value at a given node is its euclidean distance from the forecasted value vector at that point in time. The length of memory used to generate the values of the parameters must at minimum be the depth of the search tree at the point of the evaluated node.

TODO figure

3.5 Performance Parameters

Hence, to generate heuristics, we require a set of parameters to track in human performance and attempt to replicate the patterns of. I will now list each of the parameters used in my implementation for testing, and how they are evaluated. However, this part of the system can and should be flexible. While I have attempted to define a set of parameters that act rather generally and so are broadly applicable to the context of increasing musical coherence so as not to require that a user be proficient in coding, the specific choice and implementation of these parameters can be changed to suit the needs of the user, and would likely yield better results if fine-tuned to match specific goals of a performance.

3.5.1 Tempo Parameter

TODO: a tempo parameter which tracks BPM, and excludes subdivisions, eg notes approx in orders of magnitude of 2 from the BPM (or even triplets and other stuff? Test this.). Put the implementation here

3.5.2 EACH OF THE OTHER PARAMETERS GOES IN A SUB-SECTION HERE

Maybe use rhythmic stability? ([Desain and Honing, 2002](#)) Look into temporal regularity ([Large and Palmer, 2002](#))

3.6 Human Interaction

TODO: It would be really good as part of the project to include more human interaction, eg allow a human to modify parameters during the AI improvisation through a UI, and even to train new parameters so that implementing new ones doesn't require code (a little UI system for which parameters to use and how to create new ones?). This is however a stretch goal for the project, and not necessary for the core functionality. If these subsections don't end up being completed, add them into future work.

3.6.1 Performance Parameter User Interface

TODO

3.7 Training New Performance Parameters

TODO

Experiments

Tuning meta parameters section, and other adapting and fine tuning of the algorithm needs to generate results used for evaluation.

We want to have stats for a few key things, maybe each in its own subsection: 1. How close does the MCTS search get to the 'perfect heuristic' eg completely matching the exact value of all parameters based on where they were for the user and how they were moving over time. This is a measure of how well the system is able to learn the user's preferences and how well it is able to predict the user's future preferences. Use this to generate hyperparameters for the MCTS search and other algorithm elements that are optimal (do we want tuning these parameters to be in a completely different section before the evaluation instead?). Eg show how this changes with - the number of iterations in the MCTS search (node expansions) - the depth of the MCTS search, including adaptive depth options and comparing them - the exploitation vs exploration parameters in the MCTS search - how continuous MCTS is implemented, eg how 'far away' a successor needs to be to be considered a new node We also want to show metrics for each of the parameters, and individually how well they alone can be tracked and predicted.

2. Comparisons of exploration and exploitation, by looking at probability of chosen node vs average probability of any node randomly chosen in the MDN distribution. If the chosen node probability is usually higher than average, shrink how wide the MDN is as the current one is too explorative, if it is usually lower, increase the width of the MDN as the current one is too exploitative. Compare against the perceived optimal from Charles' study when only choosing randomly, I hypothesise that if the guidance of MCTS search is strong, it should learn toward wider distributions being optimal than are optimal with random selection.

Evaluation

5.1 Tuning Metaparameters

5.2 Quantitative Evaluation

NOTE: To evaluate with more MCTS evaluations in the same time frame, we can emulate a higher power computer by running the algorithm for 10x longer to produce outputs, but then compressing them to their original intended duration (run everything on 0.1x speed).

3. How well does the prediction of parameters over time actually work? Does the movement tracking of parameters lead to sensible results? Are added motifs actually reused in a sensible way? Are choruses repeated? Do parameters get wacky and completely off base after the system being left too long on its own?
4. Look in the literature for how to evaluate the quality of generated music, and how to evaluate the quality of the system's predictions. Make direct comparisons to the same system without using the MCTS search.
5. How well does the system perform in terms of computational resources? How long does it take to generate a piece of music? How much memory does it use? How much CPU does it use? How much does this change with the number of iterations in the MCTS search, the depth of the MCTS search, the exploitation vs exploration parameters in the MCTS search, and how continuous MCTS is implemented?
6. Compare the predictive power of the model, generate a test set of improvisations, and test the accuracy of predicting the next note in the improvisation both with and without the MCTS search.

5.3 Qualitative Evaluation

User trial, my own experience, a piece I've created using the system, my personal thoughts on each of the parameters and how well they function. Try to make this sem-formal: I sat down and tried 5 performances, with these 5 aims, etc etc. Comment on the previous sections points from the previous section in a more personal subjective manner, plus the following: - How well does the system perform in terms of user interaction? How easy is it to use? How intuitive is it to use? How much does the user need to interact with the system to get a good result? Note this for differing hyperparameters too.

Discussion

6.1 Limitations

6.2 Future Work

TODO: markov models and trying to get long term repetition, eg chorus repetition, to work. Could use markov models. Remove this from previous sections as its probably too much to get done for now.

TODO: if there is time, instead of using previous measurements from human performance and maintaining them, an additional RNN could be used to continuously predict the next state of the set of musical parameters.

What about back and forth or simultaneous interaction? Based on the use of similar techniques in robot path planning environments, it seems like planning the future of a piece by using MDNs to predict the actions of collaborators, while intersecting with own actions through search trees, could be a promising approach.

Conclusion

TODO

Bibliography

- ALLAN, M. AND WILLIAMS, C. K. I., 2004. Harmonising chorales by probabilistic inference. In *Proceedings of the 17th International Conference on Neural Information Processing Systems (NIPS 2004)*, 25–32. MIT Press. https://proceedings.neurips.cc/paper_files/paper/2004/file/b628386c9b92481fab68fbf284bd6a64-Paper.pdf. [Cited on page 12.]
- ANDERS, T. AND MIRANDA, E. R., 2011. Constraint programming systems for modeling music theories and composition. *ACM Computing Surveys*, 43, 4 (2011). doi:10.1145/1978802.1978809. [Cited on pages 2 and 19.]
- BENGIO, Y.; SIMARD, P. Y.; AND FRASCONI, P., 1994. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5, 2 (1994), 157–166. doi:10.1109/72.279181. [Cited on page 14.]
- BISHOP, C. M., 1994. Mixture density networks. Technical Report NCRG/97/004, Neural Computing Research Group, Aston University. [Cited on page 17.]
- BISIANI, R., 1992. Beam search. In *Encyclopedia of Artificial Intelligence*, vol. 2, 1467–1468. John Wiley and Sons, 2 edn. [Cited on page 21.]
- BROWN, N. AND SANDHOLM, T., 2019. Superhuman AI for multiplayer poker. *Science*, 365, 6456 (2019), 885–890. doi:10.1126/science.aay2400. [Cited on page 26.]
- BROWNE, C.; POWLEY, E. J.; WHITEHOUSE, D.; LUCAS, S. M.; COWLING, P. I.; ROHLFSHAGEN, P.; TAVENER, S.; LIEBANA, D. P.; SAMOTHRAKIS, S.; AND COLTON, S., 2012. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4, 1 (2012), 1–43. doi:10.1109/TCIAIG.2012.2186810. [Cited on page 23.]
- BRÜGMANN, B., 1993. Monte carlo go. Technical report, Physics Department, Syracuse University. [Cited on page 23.]
- CHEN, J.; ZHANG, C.; LUO, J.; XIE, J.; AND WAN, Y., 2020. Driving maneuvers prediction based autonomous driving control by deep monte carlo tree search. *IEEE*

Bibliography

- Transactions on Vehicular Technology*, 69, 7 (2020), 7146–7158. doi:10.1109/TVT.2020.2991584. [Cited on page 27.]
- COLLINS, N. AND D’ESCRIVÁN, J., 2012. *The Cambridge Companion to Electronic Music*. Cambridge University Press. doi:10.1017/CCOL9780521868617. [Cited on page 8.]
- COLLINS, N. M., 2007. *Towards autonomous agents for live computer music : real-time machine listening and interactive music systems*. Ph.D. thesis, University of Cambridge. <https://api.semanticscholar.org/CorpusID:26846600>. [Cited on page 10.]
- COULOM, R., 2006. Efficient selectivity and backup operators in monte-carlo tree search. In *Proceedings of the 5th International Conference on Computers and Games (CG 2006)*, 72–83. Springer International Publishing. doi:10.1007/978-3-540-75538-8\7. [Cited on page 23.]
- DANNENBERG, R. B.; THOM, B.; AND WATSON, D., 1997. A machine learning approach to musical style recognition. In *Proceedings of the 23rd International Computer Music Conference (ICMC 1997)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.1997.090>. [Cited on page 13.]
- DESAIN, P. AND HONING, H., 2002. Rhythmic stability as explanation of category size. In *Proceedings of the 7th International Conference on Music Perception & Cognition (ICMPC 2002)*, 111–115. Architecture, Media, Politics, Society and Causal Productions. <https://hdl.handle.net/11245/1.206041>. [Cited on pages 10 and 38.]
- DOLSON, M., 1989. Machine tongues XII: Neural networks. *Computer Music Journal*, 13, 3 (1989), 28–40. [Cited on page 13.]
- EBCIOĞLU, K., 1988. An expert system for harmonizing four-part chorales. *Computer Music Journal*, 12, 3 (1988), 43–51. doi:10.2307/3680335. [Cited on page 19.]
- ECK, D. AND SCHMIDHUBER, J., 2002. Finding temporal structure in music: blues improvisation with LSTM recurrent networks. In *Proceedings of the 12th IEEE Workshop on Neural Networks for Signal Processing (NNSP 2002)*, 747–756. IEEE. doi:10.1109/NNSP.2002.1030094. [Cited on page 15.]
- FERREIRA, L. N.; MOU, L.; WHITEHEAD, J.; AND LELIS, L. H. S., 2022. Controlling perceived emotion in symbolic music generation with monte carlo tree search. In *Proceedings of the 18th AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE 2022)*, vol. 25, 3763–3772. AAAI Press. doi:10.1109/TMM.2022.3165718. [Cited on pages 20 and 21.]
- FIEBRINK, R.; TRUEMAN, D.; AND COOK, P. R., 2009. A meta-instrument for interactive, on-the-fly machine learning. In *Proceedings of the 9th International Conference on New Interfaces for Musical Expression (NIME 2009)*, 280–285. Zenodo. doi:10.5281/ZENODO.1177513. [Cited on page 9.]

- FU, X.; DENG, H.; YUAN, X.; AND HU, J., 2023. Generating high coherence monophonic music using monte-carlo tree search. *IEEE Transactions on Multimedia*, 25 (2023), 3763–3772. doi:10.1109/TMM.2022.3165718. [Cited on pages 2, 20, and 21.]
- GRAVES, A., 2013. Generating sequences with recurrent neural networks. *ArXiv e-prints*, 1308.0850 (2013). doi:10.48550/arXiv.1308.0850. [Cited on page 17.]
- HA, D. AND ECK, D., 2017. A neural representation of sketch drawings. *ArXiv e-prints*, 1704.03477 (2017). doi:10.48550/arXiv.1704.03477. [Cited on page 17.]
- HADJERES, G.; PACHET, F.; AND NIELSEN, F., 2017. DeepBach: a steerable model for bach chorales generation. In *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, vol. 70, 1362–1371. Proceedings of Machine Learning Research. doi:10.48550/arXiv.1612.01010. [Cited on page 15.]
- HILLER, L. A. AND ISAACSON, L. M., 1959. *Experimental Music: Composition With an Electronic Computer*. McGraw-Hill, 2nd edn. Reprinted, Westport, Conn.: Greenwood Press, 1979. [Cited on page 19.]
- HOCHREITER, S. AND SCHMIDHUBER, J., 1997. Long short-term memory. *Neural Computation*, 9, 8 (1997), 1735–1780. doi:10.1162/NECO.1997.9.8.1735. [Cited on page 15.]
- HOKAMP, C. AND LIU, Q., 2017. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL 2017)*, 1535–1546. Association for Computational Linguistics. doi:10.18653/v1/P17-1141. [Cited on page 26.]
- HUNT, A.; WANDERLEY, M. M.; AND KIRK, R., 2000. Towards a model for instrumental mapping in expert musical interaction. In *Proceedings of the 26th International Computer Music Conference (ICMC 2000)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.2000.173>. [Cited on page 8.]
- JI, S.; YANG, X.; AND LUO, J., 2024. A survey on deep learning for symbolic music generation: Representations, algorithms, evaluations, and challenges. *ACM Computing Surveys*, 56, 1 (2024). doi:10.1145/3597493. [Cited on pages 15 and 16.]
- JOHNSON, D. D., 2017. Generating polyphonic music using tied parallel networks. In *Proceedings of the 6th International Conference on Computational Intelligence in Music, Sound, Art and Design (EvoMUSART 2017)*, 128–143. Springer International Publishing. doi:10.1007/978-3-319-55750-2_9. [Cited on page 16.]
- KARKUS, P.; HSU, D.; AND LEE, W. S., 2017. QMDP-net: Deep learning for planning under partial observability. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS 2017)*, 4705–4714. MIT Press. doi:10.48550/arXiv.1703.06692. [Cited on page 27.]

Bibliography

- KIEFER, C.; COLLINS, N.; AND FITZPATRICK, G., 2008. HCI methodology for evaluating musical controllers: A case study. In *Proceedings of the 8th International Conference on New Interfaces for Musical Expression (NIME 2008)*, 87–90. Zenodo. doi:10.5281/zenodo.1179577. [Cited on page 8.]
- KOCSIS, L. AND SZEPESVÁRI, C., 2006. Bandit based monte-carlo planning. In *Proceedings of the 17th European Conference on Machine Learning (ECML 2006)*, 282–293. Springer International Publishing. doi:10.1007/11871842_29. [Cited on pages 22 and 23.]
- LADEN, B. AND KEEFE, D. H., 1989. The representation of pitch in a neural net model of chord classification. *Computer Music Journal*, 13, 4 (1989), 12–26. doi:10.2307/3679550. [Cited on page 13.]
- LARGE, E. W. AND PALMER, C., 2002. Perceiving temporal regularity in music. *Cognitive Science*, 26, 1 (2002), 1–37. doi:10.1016/S0364-0213(01)00057-X. [Cited on page 38.]
- LAWSON, J. AND MATHEWS, M. V., 1977. Computer program to control a digital real-time sound synthesizer. *Computer Music Journal*, 1, 4 (1977), 16–21. [Cited on page 8.]
- LECUN, Y.; BENGIO, Y.; AND HINTON, G., 2015. Deep learning. *Nature*, 521, 7553 (2015), 436–444. doi:10.1038/nature14539. [Cited on page 13.]
- LEI, L.; LUO, R.; ZHENG, R.; WANG, J.; ZHANG, J.; QIU, C.; MA, L.; JIN, L.; ZHANG, P.; AND CHEN, J., 2021. KB-Tree: Learnable and continuous monte-carlo tree search for autonomous driving planning. In *Proceedings of the 34th International Conference on Intelligent Robots and Systems (IROS 2021)*, 4493–4500. IEEE Press. doi:10.1109/IROS51168.2021.9636442. [Cited on pages 25 and 27.]
- LOWERRE, B. T., 1976. *The harpy speech recognition system*. Ph.D. thesis, Carnegie Mellon University. [Cited on page 21.]
- MACHOVER, T. AND CHUNG, J. T., 1989. Hyperinstruments: Musically intelligent and interactive performance and creativity systems. In *Proceedings of the 15th International Computer Music Conference (ICMC 1989)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.1989.046>. [Cited on page 9.]
- MAO, H. H.; SHIN, T.; AND COTTRELL, G. W., 2018. DeepJ: Style-specific music generation. In *Proceedings of the 12th IEEE International Conference on Semantic Computing (ICSC 2018)*, 377–382. IEEE Computer Society. doi:10.1109/ICSC.2018.00077. [Cited on page 16.]
- MARTIN, C. P. AND TØRRESEN, J., 2018. RoboJam: A musical mixture density network for collaborative touchscreen interaction. In *Proceedings of the 7th International*

- Conference on Computational Intelligence in Music, Sound, Art and Design (EvoMUSART 2018)*, 161–176. Springer International Publishing. doi:10.1007/978-3-319-77583-8_11. [Cited on pages 2 and 17.]
- MARTIN, C. P. AND TØRRESEN, J., 2019. An interactive musical prediction system with mixture density recurrent neural networks. In *Proceedings of the 19th International Conference on New Interfaces for Musical Expression (NIME 2019)*, 260–265. Zenodo. doi:10.5281/ZENODO.3672952. [Cited on pages 2, 3, 9, 10, 17, 21, and 28.]
- MATHEWS, M. V., 1991. The radio baton and conductor program, or: Pitch, the most important and least expressive part of music. *Computer Music Journal*, 15, 4 (1991), 37–46. doi:10.2307/3681070. [Cited on page 8.]
- MCPHERSON, A., 2010. The magnetic resonator piano: Electronic augmentation of an acoustic grand piano. *Journal of New Music Research*, 39, 3 (2010), 189–202. doi:10.1080/09298211003695587. [Cited on page 9.]
- MERN, J.; YILDIZ, A.; SUNBERG, Z.; MUKERJI, T.; AND KOCHENDERFER, M. J., 2021. Bayesian optimized monte carlo planning. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI 2021)*, 11880–11887. AAAI Press. doi:10.1609/AAAI.V35I13.17411. [Cited on page 25.]
- METROPOLIS, N. AND ULAM, S., 1949. The monte carlo method. *Journal of the American Statistical Association*, 44, 247 (1949), 335–341. doi:10.2307/2280232. [Cited on page 23.]
- MOORER, J. A., 1972. Music and computer composition. *Communications of the Association for Computing Machinery*, 15, 2 (1972), 104–113. doi:10.1145/361254.361265. [Cited on page 19.]
- MOZER, M. C., 1994. Neural network music composition by prediction: Exploring the benefits of psychoacoustic constraints and multi-scale processing. *Connection Science*, 6 (1994), 247–280. doi:10.1080/09540099408915726. [Cited on page 15.]
- PACHET, F., 2002. The continuator: Musical interaction with style. In *Proceedings of the 28th International Computer Music Conference (ICMC 2002)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.2002.044>. [Cited on page 10.]
- PUCKETTE, M. S., 1997. Pure data. In *Proceedings of the 23rd International Computer Music Conference (ICMC 1997)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.1997.060>. [Cited on page 8.]
- PUCKETTE, M. S., 2002. Max at seventeen. *Computer Music Journal*, 26, 4 (2002), 31–43. doi:10.1162/014892602320991356. [Cited on page 8.]
- RENAUD, A.; CARÔT, A.; AND REBELO, P., 2007. Networked music performance: State of the art. In *Proceedings of the 30th AES International Conference on Intelligent*

Bibliography

- Audio Environments (AES 2007)*, 16–22. Audio Engineering Society. <https://aes2.org/publications/elibrary-page/?id=13914>. [Cited on page 10.]
- RON, D.; SINGER, Y.; AND TISHBY, N., 1996. The power of amnesia: Learning probabilistic automata with variable memory length. *Machine Learning*, 25 (1996), 117–149. doi:10.1023/A:1026490906255. [Cited on page 11.]
- SILVER, D.; HUANG, A.; MADDISON, C. J.; GUEZ, A.; SIFRE, L.; VAN DEN DRIESCHKE, G.; SCHRITTWIESER, J.; ANTONOGLOU, I.; PANNEERSHELVAM, V.; LANCTOT, M.; DIELEMAN, S.; AND JOHN NHAM, D. G.; KALCHBRENNER, N.; SUTSKEVER, I.; LILLICRAP, T. P.; LEACH, M.; KAVUKCUOGLU, K.; GRAEPEL, T.; AND HASSABIS, D., 2016. Mastering the game of go with deep neural networks and tree search. *Nature*, 529 (2016), 484–489. doi:10.1038/nature16961. [Cited on pages 26 and 27.]
- SILVER, D.; HUBERT, T.; SCHRITTWIESER, J.; ANTONOGLOU, I.; LAI, M.; GUEZ, A.; LANCTOT, M.; SIFRE, L.; KUMARAN, D.; GRAEPEL, T.; LILLICRAP, T.; SIMONYAN, K.; AND HASSABIS, D., 2018. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362, 6419 (2018), 1140–1144. doi:10.1126/science.aar6404. [Cited on page 26.]
- SILVER, D.; SCHRITTWIESER, J.; SIMONYAN, K.; ANTONOGLOU, I.; HUANG, A.; GUEZ, A.; HUBERT, T.; BAKER, L.; LAI, M.; BOLTON, A.; CHEN, Y.; LILLICRAP, T. P.; HUI, F.; SIFRE, L.; VAN DEN DRIESCHKE, G.; GRAEPEL, T.; AND HASSABIS, D., 2017. Mastering the game of go without human knowledge. *Nature*, 550 (2017), 354–359. doi:10.1038/nature24270. [Cited on page 26.]
- SIMON, I.; MORRIS, D.; AND BASU, S., 2008. MySong: automatic accompaniment generation for vocal melodies. In *Proceedings of the 28th Conference on Human Factors in Computing Systems (CHI 2008)*, 725–734. Association for Computing Machinery. doi:10.1145/1357054.1357169. [Cited on page 12.]
- STEEDMAN, M. J., 1984. A generative grammar for jazz chord sequences. *Music Perception: An Interdisciplinary Journal*, 2, 1 (1984), 52–77. doi:10.2307/40285282. [Cited on page 19.]
- STEINBISS, V.; TRAN, B.; AND NEY, H., 1994. Improvements in beam search. In *Proceedings of the 3rd International Conference on Spoken Language Processing (ICSLP 1994)*, 2143–2146. International Science Community Association. doi:10.21437/ICSLP.1994-538. [Cited on page 21.]
- SUTSKEVER, I.; VINYALS, O.; AND LE, Q. V., 2014. Sequence to sequence learning with neural networks. In *Proceedings of the 28th International Conference on Neural Information Processing Systems (NIPS 2014)*, vol. 2, 3104–3112. MIT Press. [dl.acm.org/doi/10.5555/2969033.2969173](https://doi.org/10.5555/2969033.2969173). [Cited on pages 22 and 26.]
- TAKASE, H. AND SHIRAMATSU, S., 2020. Support system for improvisational ensemble based on long short-term memory using smartphone sensor. In *Proceedings of the 20th*

- International Conference on New Interfaces for Musical Expression (NIME 2020)*, 394–398. Zenodo. doi:10.5281/ZENODO.4813434. [Cited on page 8.]
- TODD, P. M., 1989. A connectionist approach to algorithmic composition. *Computer Music Journal*, 13, 4 (1989), 27–43. doi:10.2307/3679551. [Cited on page 14.]
- WANG, G.; BRYAN, N. J.; OH, J.; AND HAMILTON, R., 2009. Stanford laptop orchestra (SLOrk). In *Proceedings of the 35th International Computer Music Conference (ICMC 2009)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.2009.14>. [Cited on page 10.]
- WEINBERG, G.; BLOSSER, B.; MALLIKARJUNA, T.; AND RAMAN, A., 2009. The creation of a multi-human, multi-robot interactive jam session. In *Proceedings of the 9th International Conference on New Interfaces for Musical Expression (NIME 2009)*, 70–73. Zenodo. doi:10.5281/ZENODO.1177705. [Cited on page 10.]
- WEINGERTNER, P.; HO, M.; TIMOFEEV, A.; AUBERT, S.; AND PITA-GIL, G., 2020. Monte carlo tree search with reinforcement learning for motion planning. In *Proceedings of the 23rd International Conference on Intelligent Transportation Systems (ITSC 2020)*, 1–7. IEEE Press. doi:10.1109/ITSC45102.2020.9294697. [Cited on page 27.]
- WU, S.; DONAHUE, C.; WATANABE, S.; AND BRYAN, N. J., 2024. Music controlnet: Multiple time-varying controls for music generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32 (2024), 2692–2703. doi:10.1109/TASLP.2024.3399026. [Cited on page 9.]
- WU, Y.; SCHUSTER, M.; CHEN, Z.; LE, Q. V.; NOROUZI, M.; MACHEREY, W.; KRIKUN, M.; CAO, Y.; GAO, Q.; MACHEREY, K.; KLINGNER, J.; SHAH, A.; JOHNSON, M.; LIU, X.; KAISER, L.; GOUWS, S.; KATO, Y.; KUDO, T.; KAZAWA, H.; STEVENS, K.; KURIAN, G.; PATIL, N.; WANG, W.; YOUNG, C.; SMITH, J.; RIESA, J.; RUDNICK, A.; VINYALS, O.; CORRADO, G.; HUGHES, M.; AND DEAN, J., 2016. Google’s neural machine translation system: Bridging the gap between human and machine translation. *ArXiv e-prints*, 1609.08144 (2016). doi:10.48550/arXiv.1609.08144. [Cited on page 26.]
- YEE, T.; LISÝ, V.; AND BOWLING, M. H., 2016. Monte carlo tree search in continuous action spaces with execution uncertainty. In *Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI 2016)*, 690–697. AAAI Press. [Cited on page 25.]
- ZHANG, Z., 2012. Microsoft kinect sensor and its effect. *IEEE Multimedia*, 19, 2 (2012), 4–10. doi:10.1109/MMUL.2012.24. [Cited on page 8.]
- ZICARELLI, D., 1987. M and jam factory. *Computer Music Journal*, 11, 4 (1987), 13–29. doi:10.2307/3680237. [Cited on page 11.]